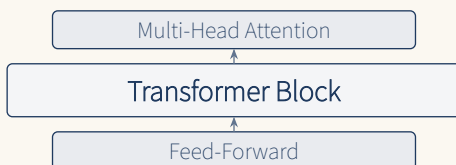


Make LLM basic

토큰나이저부터 트랜스포머까지
처음부터 직접 만드는 대형 언어 모델

dirmich

Highmaru Press



Contents

용어집	ii
머리말	v
1 LLM이란 무엇인가	1
1.1 언어 모델이란	1
1.2 직관: 언어 모델을 어떻게 생각할 것인가	1
1.3 LLM의 발전 역사	2
1.3.1 통계적 언어 모델 (1990년대~2000년대)	2
1.3.2 신경망 언어 모델 (2003~2017)	2
1.3.3 트랜스포머의 등장 (2017)	3
1.3.4 GPT 시리즈와 스케일링 법칙 (2018~현재)	3
1.3.5 정렬의 등장 (2022~현재)	4
1.4 LLM의 주요 응용	4
1.4.1 자연어 처리 전통 태스크	4
1.4.2 생성 태스크	4
1.4.3 전문 분야	4
1.4.4 멀티모달	4
1.5 왜 직접 만들어야 하는가	4
1.6 이 책이 만들 모델	5
1.7 학습 로드맵	5
1.8 요약	5
1.9 연습 문제	6
2 개발 환경 세팅	7
2.1 필요한 도구	7
2.2 PyTorch 설치	7
2.3 저장소 구조	7
2.4 첫 번째 예제: 텐서 다루기	8
2.5 재현성 확보	9
2.6 단위 테스트 실행	9
2.7 요약	9
3 토큰라이저 만들기	10
3.1 왜 토큰라이저가 필요한가	10
3.1.1 세 가지 접근 비교	10
3.2 특수 토큰	10

3.3	베이스 클래스	11
3.4	BPE 토크나이저	12
3.4.1	알고리즘 직관	12
3.4.2	왜 바이트 단위인가?	13
3.4.3	구현: 학습	13
3.4.4	인코딩	14
3.4.5	전처리: 공백 처리	15
3.4.6	디코딩: 바이트 복원	15
3.5	Unigram 토크나이저	15
3.5.1	직관	16
3.5.2	알고리즘	16
3.5.3	Viterbi 디코딩	16
3.6	BPE vs Unigram 비교	17
3.7	토크나이저 테스트	17
3.8	실습: 실제 말뭉치로 토크나이저 학습	18
3.9	요약	18
3.10	연습 문제	18
4	임베딩과 포지셔널 인코딩	20
4.1	왜 임베딩이 필요한가	20
4.2	임베딩의 수학적 정의	20
4.3	Token Embedding 구현	20
4.3.1	padding_idx의 역할	21
4.4	포지셔널 인코딩	21
4.4.1	직관: 왜 순서가 중요한가?	21
4.4.2	Sinusoidal 위치 인코딩 (Vaswani et al. 2017)	22
4.4.3	학습 가능한 위치 임베딩 (GPT-2 스타일)	23
4.5	임베딩 합성	23
4.5.1	왜 스케일링이 필요한가?	23
4.6	Layer Normalization 소개	24
4.7	테스트	24
4.8	실습: 임베딩 시각화	24
4.9	요약	25
4.10	연습 문제	25
5	어텐션 메커니즘	26
5.1	직관: 어텐션이란 무엇인가	26
5.1.1	직관적 비유: 도서관 검색	26
5.2	Q, K, V는 어디서 오는가	26
5.3	스케일링: 왜 $\sqrt{d_k}$ 로 나누는가?	27
5.4	Scaled Dot-Product Attention 구현	27
5.5	인과 마스크 (Causal Mask)	28
5.5.1	왜 $-\infty$ 인가?	28
5.6	Multi-Head Attention	28
5.6.1	아이디어	28

5.6.2	왜 헤드를 나누는가?	29
5.6.3	구현	29
5.7	인과성 테스트	30
5.8	Self-Attention vs Cross-Attention	30
5.8.1	교차 어텐션은 어떻게 다른가?	31
5.9	어텐션 계산량 분석	31
5.10	실습: 어텐션 가중치 시각화	31
5.11	요약	32
5.12	연습 문제	32
6	트랜스포머 블록 조립하기	33
6.1	트랜스포머 블록의 구조	33
6.2	Feed-Forward Network (FFN)	34
6.2.1	직관: 왜 확장-축소인가?	34
6.3	GELU vs ReLU	34
6.3.1	왜 GELU가 더 나은가?	35
6.4	잔차 연결 (Residual Connection)	35
6.4.1	직관: 왜 잔차가 도움인가?	35
6.5	Pre-LN vs Post-LN	36
6.5.1	Post-LN (Vaswani 원본)	36
6.5.2	Pre-LN (GPT-2 스타일)	36
6.5.3	왜 Pre-LN이 더 안정적인가?	36
6.6	잔차 스케일링	37
6.6.1	왜 $2N$ 인가?	37
6.7	블록 테스트	37
6.8	트랜스포머 블록의 계산량	38
6.9	요약	38
6.10	연습 문제	38
7	GPT 모델 완성과 학습 루프	39
7.1	GPT 모델 구조	39
7.2	가중치 타이 (Weight Tying)	39
7.3	Forward Pass	40
7.4	데이터셋: 슬라이딩 윈도우	41
7.5	손실 함수: Cross-Entropy	41
7.5.1	Label Smoothing (선택)	42
7.6	옵티마이저: AdamW	42
7.7	학습률 스케줄러	43
7.7.1	왜 warmup이 필요한가?	43
7.8	그래디언트 클리핑	43
7.9	트레이너	43
7.10	Perplexity	44
7.11	체크포인트 저장/로드	44
7.12	요약	45
7.13	연습 문제	45

8	텍스트 생성 — 샘플링 전략과 디코딩	46
8.1	자기회귀 생성의 원리	46
8.2	Greedy 샘플링	46
8.3	Temperature 샘플링	47
8.4	Top-K 샘플링	47
8.4.1	Top-K의 직관	48
8.5	Top-P (Nucleus) 샘플링	48
8.5.1	Top-K vs Top-P	49
8.6	샘플링 전략 비교	49
8.7	생성 함수 구현	49
8.8	컨텍스트 길이 제한	50
8.9	샘플러 테스트	50
8.10	실습: 다양한 샘플링 비교	50
8.11	고급 디코딩 기법 (소개)	51
8.11.1	Beam Search	51
8.11.2	Speculative Decoding	51
8.11.3	Contrastive Search	51
8.12	요약	51
8.13	연습 문제	51
9	정리와 다음 단계	52
9.1	1권 요약	52
9.1.1	구현한 것	52
9.1.2	테스트	52
9.1.3	독자가 가진 것	52
9.2	1권의 한계	53
9.3	2권에서 다룰 것	53
9.3.1	분산 학습 (2~4장)	53
9.3.2	최신 아키텍처 (5~6장)	53
9.3.3	양자화와 추론 최적화 (7장)	53
9.3.4	정렬과 파인튜닝 (8장)	53
9.3.5	데이터 파이프라인 (9장)	54
9.4	추천 학습 경로	54
9.5	마지막으로	54
A	수학 기초 보충	55
A.1	벡터와 행렬	55
A.2	Softmax	55
A.3	Cross-Entropy	55
A.4	연쇄 법칙과 Backpropagation	56
A.5	행렬 미분	56
A.6	정규 분포	56
A.7	정보 이론 기초	56
A.8	최적화 기초	56
B	코드 전체 목록	58

B.1	프로젝트 구조	58
B.2	테스트 실행	58
B.3	빠른 시작 예제	59
B.4	모듈별 API 요약	60
B.5	설정 데이터클래스	60
B.6	라이선스	60
C	자주 묻는 질문과 함정	61
C.1	환경 관련	61
C.1.1	Q: PyTorch 설치 후 CUDA가 인식되지 않아요	61
C.1.2	Q: 학습이 너무 느려요	61
C.2	토큰나이저 관련	61
C.2.1	Q: BPE로 학습한 토큰나이저가 “안녕”을 이상하게 토큰화해요	61
C.2.2	Q: encode/decode 라운드트립이 실패해요	62
C.2.3	Q: 특수 토큰 ID가 바뀌면 어떻게 되나요	62
C.3	모델 관련	62
C.3.1	Q: 어텐션 출력이 NaN이 나와요	62
C.3.2	Q: 모델이 같은 단어를 반복해요	62
C.3.3	Q: 컨텍스트 길이를 초과하면 에러가 나요	62
C.4	학습 관련	62
C.4.1	Q: 손실이 줄어들지 않아요	63
C.4.2	Q: 학습 초기에 손실이 발산해요	63
C.4.3	Q: 체크포인트를 로드하면 다른 결과가 나와요	63
C.5	분산 학습 관련 (2권 미리보기)	63
C.5.1	Q: DDP로 학습하면 속도가 오히려 느려요	63
C.5.2	Q: FSDP에서 OOM이 나요	63
C.6	디버깅 팁	63
C.6.1	단계별로 검증하라	64
C.6.2	shape를 항상 출력하라	64
C.6.3	작은 입력으로 시작하라	64
C.7	성능 최적화 팁	64
C.7.1	혼합 정밀도 사용	64
C.7.2	torch.compile	64
C.7.3	프로파일링	64
D	연습 문제 해답	66
D.1	1장 해답	66
D.2	3장 해답	66
D.3	4장 해답	67
D.4	5장 해답	67
D.5	6장 해답	68
D.6	7장 해답	68
D.7	8장 해답	68
E	고급 주제 미리보기	70
E.1	분산 학습	70

E.2	최신 아키텍처	70
E.3	양자화와 추론 최적화	70
E.4	정렬과 파인튜닝	71
E.5	데이터 파이프라인	71
E.6	2권 학습 준비	71
E.7	추천 학습 경로	71

용어집

이 용어집은 본 책에서 사용하는 주요 용어를 알파벳순으로 정리한 것이다. 각 항목은 짧은 정의와 함께 해당 장 번호를 표시하여 독자가 쉽게 참조할 수 있도록 구성했다. 처음 책을 읽을 때는 가볍게 훑어보고, 본문을 읽다가 모르는 용어가 나오면 다시 돌아와 참조하기 바란다.

Activation (활성값)

신경망의 각 층을 통과한 후 출력되는 값. 레이어의 출력 텐서를 의미하며, 다음 층의 입력이 된다. 참고: 4장

AdamW

Adam 옵티마이저에 weight decay를 결합한 최적화 알고리즘. 현대 LLM 학습의 사실상 표준. 참고: 7장

Attention (어텐션)

시퀀스의 각 위치가 다른 모든 위치에 얼마나 주의를 기울일지 학습하는 메커니즘. 트랜스포머의 핵심. 참고: 5장

Backpropagation

신경망의 손실을 각 파라미터로 미분하여 그래디언트를 계산하는 알고리즘. 연쇄 법칙(chain rule)에 기반. 참고: 7장

BPE (Byte Pair Encoding)

바이트 단위로 시작하여 가장 자주 등장하는 쌍을 병합해 나가는 토큰나이저 알고리즘. GPT-2, GPT-3 등이 사용. 참고: 3장

Causal Mask (인과 마스크)

자기회귀 모델에서 미래 토큰을 보지 못하도록 가리는 하삼각 마스크. 참고: 5장

Context Length (컨텍스트 길이)

모델이 한 번에 처리할 수 있는 최대 토큰 수. GPT-2는 1024, GPT-4는 128K. 참고: 7장

Cross-Entropy Loss (교차 엔트로피 손실)

분류 문제에서 예측 확률 분포와 정답 분포 사이의 차이를 측정하는 손실 함수. 언어 모델링의 표준 손실. 참고: 7장

Decoder (디코더)

트랜스포머에서 시퀀스를 생성하는 부분. GPT는 디코더만 사용하는 구조. 참고: 6장

Dropout

학습 시 일부 뉴런을 무작위로 0으로 만들어 과적합을 방지하는 정규화 기법. 참고: 4장

Embedding (임베딩)

이산적인 토큰 ID를 연속적인 밀집 벡터로 변환하는 과정. 또는 그 벡터 자체. 참고: 4장

Feed-Forward Network (FFN, 순방향 신경망)

트랜스포머 블록에서 어텐션 다음에 적용되는 위치별 완전연결 신경망. 보통 2개의 선형 레이어 + 활성화 함수. 참고: 6장

GELU (Gaussian Error Linear Unit)

ReLU의 부드러운 버전. 가우시안 함수를 사용하여 0 주변에서 부드럽게 전환. GPT 모델이 사용. 참고: 6장

Gradient (그래디언트)

손실 함수의 각 파라미터에 대한 미분값. 최적화 방향을 가리킴. 참고: 7장

Greedy Decoding (탐욕 디코딩)

각 스텝에서 가장 확률이 높은 토큰만 선택하는 생성 전략. 빠르지만 단조로운 출력. 참고: 8장

Head (헤드)

멀티헤드 어텐션에서 독립적으로 어텐션을 수행하는 단위. 참고: 5장

Layer Normalization (레이어 정규화)

텐서의 마지막 차원을 따라 평균과 분산을 정규화하는 기법. 트랜스포머 학습 안정화에 필수. 참고: 4장, 6장

Learning Rate (학습률)

그래디언트의 스텝 크기를 결정하는 하이퍼파라미터. 너무 크면 발산, 너무 작으면 느린 수렴. 참고: 7장

Logits (로짓)

softmax를 적용하기 전의 원시 출력 값. 보통 모델의 마지막 선형 레이어 출력. 참고: 7장

Loss Function (손실 함수)

모델의 예측과 정답 사이의 차이를 스칼라로 측정하는 함수. 학습의 최적화 대상. 참고: 7장

Multi-Head Attention (멀티헤드 어텐션)

여러 개의 어텐션 헤드를 병렬로 수행하여 다양한 패턴을 동시에 학습. 참고: 5장

Parameter (파라미터)

모델이 학습하는 가중치. 선형 레이어의 W , b 등. 참고: 7장

Perplexity (퍼플렉서티, 혼란도)

언어 모델의 평가 지표. $PPL = \exp(\text{loss})$. 낮을수록 좋은 모델. 참고: 7장

Positional Encoding (위치 인코딩)

트랜스포머에 순서 정보를 주입하는 방법. sin/cos 또는 학습 가능한 임베딩. 참고: 4장

Pre-LN / Post-LN

레이어 정규화의 위치. Pre-LN은 어텐션/FFN 전에, Post-LN은 후에 적용. Pre-LN이 학습 안정성 우수. 참고: 6장

Residual Connection (잔차 연결)

입력을 출력에 더하는 연결. $y = x + f(x)$. 깊은 네트워크 학습에 필수. 참고: 6장

Scaled Dot-Product Attention

$Q \cdot K^T / \sqrt{d_k}$ 형태의 어텐션. 트랜스포머의 가장 기본 연산. 참고: 5장

Self-Attention (자기 어텐션)

같은 시퀀스 내의 토큰들 사이의 어텐션. 트랜스포머의 핵심. 참고: 5장

Softmax

벡터를 확률 분포로 변환하는 함수. $\text{softmax}(x_i) = e^{x_i} / \sum_j e^{x_j}$. 참고: 5장

Temperature (온도)

샘플링 시 분포의 날카로움을 조절하는 파라미터. 낮을수록 결정론적, 높을수록 무작위. 참고: 8장

Token (토큰)

텍스트를 분할한 최소 단위. 단어, 부분단어, 또는 바이트일 수 있음. 참고: 3장

Tokenizer (토크나이저)

텍스트를 토큰 ID 시퀀스로 변환하고, 그 반대 과정도 수행하는 모듈. 참고: 3장

Top-K Sampling

다음 토큰 후보 중 확률이 가장 높은 K개만 고려하여 샘플링하는 전략. 참고: 8장

Top-P (Nucleus) Sampling

누적 확률이 P가 될 때까지의 토큰만 고려하는 샘플링 전략. 분포 형태에 따라 적응적. 참고: 8장

Transformer (트랜스포머)

Vaswani et al. (2017)이 제안한 어텐션 기반 신경망 구조. 현대 LLM의 기반. 참고: 6장

Unigram Tokenizer

각 토큰에 확률을 할당하고 EM으로 학습하는 토크나이저. SentencePiece가 사용. 참고: 3장

Viterbi Algorithm

동적 계획법으로 최적 분할을 찾는 알고리즘. Unigram 토크나이저의 디코딩에 사용. 참고: 3장

Vocabulary (어휘)

토크나이저가 인식하는 모든 토큰의 집합. 어휘 크기는 보통 1만 10만. 참고: 3장

Weight Tying (가중치 타이)

입력 임베딩과 출력 레이어의 가중치를 공유하여 파라미터 수를 줄이는 기법. 참고: 7장

머리말

왜 이 책을 썼는가

2022년 말 ChatGPT가 공개된 이후, 대형 언어 모델(LLM)은 인공지능 분야의 가장 뜨거운 주제가 되었다. 수많은 개발자와 연구자가 HuggingFace Transformers를 통해 이미 만들어진 모델을 다운로드하고, 파인튜닝하고, 서비스에 통합하는 방법을 빠르게 익혔다. 하지만 “이 모델이 내부적으로 어떻게 동작하는가?”라는 질문에 자신 있게 답할 수 있는 사람은 여전히 소수다.

이 책은 그 질문에 답하기 위해 쓰였다. 토큰라이저를 직접 만들고, 어텐션을 손으로 구현하고, 트랜스포머 블록을 조립하고, 학습 루프를 돌려보면서 LLM의 뼈대를 이해하는 것이 이 책의 목표다. 추상화된 라이브러리를 쓰지 않고 PyTorch의 기본 연산만으로 처음부터 끝까지 직접 만든다. 그 과정에서 겪는 모든 디테일이 결국 LLM을 “이해”하는 길이다.

이 책의 특징

이 책은 단순히 이론을 나열하지 않는다. **코드를 먼저 만들고, 단위 테스트로 검증한 뒤, 그 과정을 책으로 풀어낸다.** 책에 등장하는 모든 Python 코드는 실제로 실행되며, pytest 기반 단위 테스트를 통과한 코드만 수록되었다. 독자는 GitHub 저장소에서 전체 코드를 다운로드하여 직접 실행하고 수정하며 학습할 수 있다.

각 장은 이전 장에서 구현한 모듈을 import하여 사용한다. 예를 들어 4장의 임베딩 모듈은 3장의 토큰라이저를 사용하고, 5장의 어텐션은 4장의 임베딩을 사용한다. 이 “쌓아가는 구조”를 통해 독자는 LLM이 어떻게 조립되는지를 단계적으로 체감하게 된다. 수학 수식은 직관적 설명 다음에 등장하며, 초보자도 흐름을 놓치지 않도록 배려했다.

누가 읽어야 하는가

이 책의 주 독자는 Python 기초 문법을 알고 있으나 딥러닝은 처음인 개발자다. HuggingFace Transformers를 써본 적은 있으나 내부 동작을 모르는 엔지니어, “Attention Is All You Need” 논문을 읽어보았으나 코드로 구현해본 적은 없는 연구자도 포함된다. 미적분과 기초 선형대수를 가정하되, 행렬 미분 등의 고급 주제는 부록에서 보충 설명한다.

2026년 여름
dirmich

감사의 글

이 책을 쓰는 동안 수많은 오픈소스 프로젝트와 논문의 도움을 받았다. 특히 Andrej Karpathy의 nanoGPT, HuggingFace Tokenizers, 그리고 PyTorch 개발팀의 훌륭한 문서에 깊은 감사를 전한다. 이들은 LLM의 민주화에 헌신한 선구자들이다.

또한 이 책의 코드가 기반을 둔 수많은 연구자들에게 감사드린다. Ashish Vaswani와 트랜스포머 원논문의 공동 저자들은 이 모든 것의 출발점을 만들어주었다. Alec Radford와 OpenAI 팀은 GPT 시리즈를 통해 스케일링의 가능성을 보여주었다. Tianyi Zhang과 Jason Wei는 instruction tuning과 정렬의 기반을 닦았다.

가족과 친구들에게도 감사를 전한다. 글을 쓰는 동안 묵묵히 기다려준 이들의 인내가 없었다면 이 책은 완성되지 않았을 것이다.

마지막으로, 이 책을 읽는 모든 독자에게. 여러분의 호기심과 열정이 LLM의 미래를 만든다. 직접 만들고, 실험하고, 실패하고, 다시 시도하라. 그것이 학습의 본질이다.

dirmich

Highmaru Press

Chapter 1

LLM이란 무엇인가

이 장에서는 대형 언어 모델(Large Language Model, LLM)이 무엇인지, 어떻게 발전해 왔는지, 그리고 왜 직접 만들어보는 것이 가치 있는지를 살펴본다. 코드보다 개념에 집중하므로, 딥러닝을 처음 접하는 독자도 부담 없이 읽을 수 있다.

1.1 언어 모델이란

언어 모델(language model)은 단어(또는 토큰)의 시퀀스에 확률을 할당하는 통계적 모델이다. 쉽게 말해, “이 문장이 자연스러운가?”를 확률로 판단하는 함수 $P(w_1, w_2, \dots, w_n)$ 다. 예를 들어 “고양이가 마우스를 쫓는다”는 자연스럽지만 “마우스가 고양이를 쫓는다”는 문맥에 따라 덜 자연스러울 수 있다. 언어 모델은 이런 확률을 학습한다.

역사적으로 언어 모델은 N-gram 모델에서 시작해 신경망 기반 모델(RNN, LSTM)을 거쳐, 2017년 트랜스포머(Transformer)의 등장과 함께 비약적으로 발전했다. 특히 “다음 토큰 예측(next token prediction)”이라는 단순한 목표로 인터넷 규모의 텍스트를 학습한 모델들이 놀라운 언어 이해 능력을 보여주면서, 오늘날의 LLM 시대가 열렸다.

다음 토큰 예측

LLM의 학습 목표는 단순하다: 지금까지의 토큰들을 보고 다음 토큰을 예측하는 것. 수식으로 쓰면

$$P(w_t \mid w_1, w_2, \dots, w_{t-1})$$

이 조건부 확률을 정확히 추정하는 것이 언어 모델의 유일한 목표다. 이 단순한 목표가 대규모 데이터와 결합했을 때 질문 응답, 코드 작성, 번역 등 복잡한 능력이 자연스럽게 등장한다.

왜 이렇게 단순한 목표가 강력한가? 언어를 “이해”하려면 문법, 상식, 추론, 사실 지식 등을 모두 갖춰야 한다. 다음 토큰을 정확히 예측하려면 모델은 이 모든 것을 암묵적으로 학습해야 한다. “지구는 태양 주위를 도는 ...” 다음에 “행성”을 예측하려면 천문학 상식이 필요하고, “1+1=” 다음에 “2”를 예측하려면 산술이 필요하다.

1.2 직관: 언어 모델을 어떻게 생각할 것인가

언어 모델을 처음 접하는 독자를 위해 비유를 들어보자. 언어 모델은 “엄청나게 많은 책을 읽은 사람”과 비슷하다. 이 사람은 모든 문장의 “다음 단어”를 맞추는 훈련을 받았다. 훈련이 끝나면 그 사람은 질문을 받았을 때 가장 자연스러운 다음 단어들을 이어붙여 답변을 “생성”한다.

물론 진짜 사람과는 다르다. 언어 모델은 “이해”한다기보다 “패턴을 매칭”한다. 하지만 충분히 많은 패턴을 학습하면 겉보기에 이해하는 것처럼 보인다. 이것이 바로 GPT-4 같은 모델이 놀라운 답변을 내놓는 원리다.

언어 모델은 “이해”하는가?

이 질문은 철학적 논쟁거리다. 기능주의(functionalism) 관점에서 “입력에 대해 적절히 반응하면 이해한다”고 볼 수 있고, 반대로 “내부 표상이 없으면 이해가 아니다”라고 볼 수도 있다. 이 책에서는 철학적 논쟁보다는 “어떻게 동작하는가”에 집중한다. 철학에 관심이 있다면 Searle의 “중국어 방” 사고실험을 찾아보라.

1.3 LLM의 발전 역사

1.3.1 통계적 언어 모델 (1990년대~2000년대)

초기 언어 모델은 N-gram이라는 단순한 통계 기법을 사용했다. “직전 N-1개의 단어를 보고 다음 단어의 확률을 추정”하는 방식이다. 예를 들어 “the cat sat on the” 다음에 올 단어로 “mat”이 “banana”보다 확률이 높다는 것을 말뭉치 통계로 학습한다.

```
1 # N-gram 언어 모델의 핵심 아이디어 단순화()
2 from collections import Counter
3
4 corpus = "the cat sat on the mat the cat ran".split()
5 # 2-gram (bigram) 카운트
6 bigrams = [(corpus[i], corpus[i+1]) for i in range(len(corpus)-1)]
7 counts = Counter(bigrams)
8 # P("mat" | "the") 추정
9 p_mat_given_the = counts[("the", "mat")] / sum(1 for w1, w2 in bigrams if w1 == "the")
10 print(f"P(mat|the) = {p_mat_given_the:.2f}")
```

하지만 N-gram은 심각한 한계가 있었다. 문맥을 짧게만 볼 수 있고(N이 보통 3~5), 학습하지 않은 조합에는 0의 확률을 할당하는 “희소성(sparsity)” 문제가 있었다. 이를 보완하기 위해 smoothing 기법이 개발되었지만 근본적 해결은 아니었다.

또 다른 문제는 “단어의 의미”를 포착하지 못한다는 것이었다. “개”와 “강아지”는 비슷한 의미지만 N-gram에서는 완전히 다른 단어로 취급된다. 이를 해결하기 위해 단어를 분산 표현(distributed representation)으로 바꾸는 신경망 기반 모델이 등장했다.

1.3.2 신경망 언어 모델 (2003~2017)

Bengio et al. (2003)은 최초로 신경망 기반 언어 모델을 제안했다. 단어를 저차원 밀집 벡터(임베딩)로 표현하고, 이 임베딩들을 신경망에 넣어 다음 단어 확률을 예측하는 방식이었다. 이후 RNN(Recurrent Neural Network)과 LSTM(Long Short-Term Memory)이 등장하면서 더 긴 문맥을 처리할 수 있게 되었다.

Word2Vec과 임베딩

Mikolov et al. (2013)의 Word2Vec은 단어를 100~300차원 벡터로 표현하면서 “비슷한 의미의 단어는 벡터 공간에서 가깝다”는 것을 보였다. 유명한 예: $\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$. 이 아이디어는 4장에서 구현할 임베딩의 기반이다.

RNN의 한계

RNN은 이전 시점의 은닉 상태를 현재 시점에 전달하는 구조다. 이론적으로는 무한한 문맥을 처리할 수 있지만, 실제로는 역전파 시 그래디언트가 사라지거나 폭발하는 **van-**

ishing/exploding gradient 문제가 발생한다. 이로 인해 실제 처리 가능한 문맥 길이는 수십~수백 토큰에 불과했다. 또한 순차 처리만 가능해 GPU 병렬화에 비효율적이었다. LSTM(Long Short-Term Memory)은 게이트 메커니즘으로 이 문제를 부분적으로 완화했지만, 근본적 해결은 아니었다. 100단어 이상의 문맥은 여전히 어려웠다.

1.3.3 트랜스포머의 등장 (2017)

2017년 Vaswani et al.이 발표한 “Attention Is All You Need”는 언어 모델의 판도를 완전히 바꿨다. RNN 없이 어텐션(attention) 메커니즘만으로 시퀀스를 처리하는 트랜스포머(Transformer) 구조를 제안했다. 트랜스포머의 핵심 혁신은 두 가지다.

1. **병렬 처리**: RNN처럼 순차적으로 처리하지 않고, 시퀀스의 모든 위치를 동시에 처리할 수 있어 GPU 활용도가 극대화되었다. 이는 학습 속도를 수십 배 향상시켰다.
2. **장거리 의존성**: 어텐션은 시퀀스의 모든 위치 쌍 사이의 직접 연결을 계산하므로, 거리가 먼 토큰 간의 관계도 잘 포착한다. 1000단어 떨어진 단어라도 한 번의 어텐션으로 참조 가능하다.

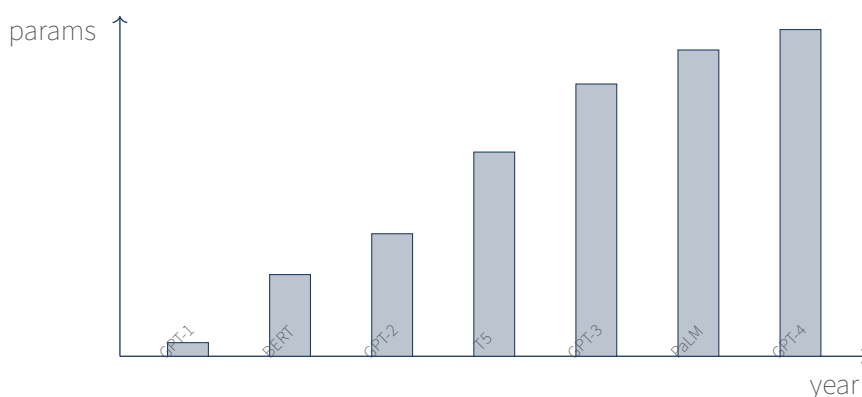


Figure 1.1: LLM 파라미터 수의 발전 추이 (로그 스케일, 대략적). GPT-1 (2018)은 1.1억 개, GPT-4 (2023)는 추정 1.7조 개.

1.3.4 GPT 시리즈와 스케일링 법칙 (2018~현재)

OpenAI의 GPT 시리즈는 트랜스포머 디코더 구조를 사용해 대규모 텍스트로 사전학습(pre-training)하는 접근을 취했다. GPT-1 (2018)은 1억 1,700만 파라미터였지만, GPT-2 (2019)는 15억, GPT-3 (2020)는 1,750억 파라미터로 급성장했다. 이 과정에서 모델 크기, 데이터 크기, 연산량을 키우면 성능이 예측 가능하게 향상된다는 **스케일링 법칙(scaling law)**이 발견되었다.

Kaplan et al. (2020)의 스케일링 법칙은 손실이 모델 크기 N , 데이터 크기 D , 연산량 C 에 대해 멱법칙(power law)을 따른다는 것을 보였다:

$$\mathcal{L}(N, D, C) \approx A \cdot N^{-\alpha_N} + B \cdot D^{-\alpha_D} + E \cdot C^{-\alpha_C} + L_\infty$$

이 법칙의 실용적 시사점은 명확하다: 모델을 키우려면 데이터도 함께 키워야 한다. 100억 파라미터 모델을 10억 토큰으로 학습하면 과적합된다. Hoffmann et al. (2022)의 Chinchilla 논문은 파라미터 N 당 약 20개의 토큰이 이상적이라고 제안했다.

1.3.5 정렬의 등장 (2022~현재)

GPT-3 (2020)는 1,750억 파라미터로 강력했지만, 사용자 질문에 도움이 되는 답변을 하지는 않았다. 그냥 “다음 토큰”을 예측했을 뿐이다. 2022년 ChatGPT가 혁신적이었던 이유는 모델 크기가 아니라 **정렬(alignment)**에 있었다.

인간 피드백을 활용한 강화학습(RLHF)이 GPT-3를 “쓸모있는 챗봇”으로 변환시킨 것이다. 이후 Direct Preference Optimization (DPO), Constitutional AI 등 더 단순하고 효율적인 정렬 기법이 등장했다. 2권 8장에서 이 기법들을 직접 구현한다.

1.4 LLM의 주요 응용

LLM은 다양한 분야에서 활용되고 있다:

1.4.1 자연어 처리 전통 태스크

- **기계 번역**: Google Translate, DeepL 등이 트랜스포머 기반.
- **요약**: 긴 문서를 짧게 요약. 뉴스, 논문, 회의록 등.
- **감성 분석**: 텍스트의 긍정/부정 판별. 리뷰, 소셜미디어 분석.
- **개체명 인식**: 텍스트에서 사람, 장소, 조직 등 추출.

1.4.2 생성 태스크

- **대화**: ChatGPT, Claude, Gemini 등.
- **코드 작성**: GitHub Copilot, Cursor. Python, JavaScript 등 다양한 언어.
- **창작**: 소설, 시, 시나리오. AI 작가 도구.
- **이메일/문서 작성**: 비즈니스 이메일, 보고서 초안.

1.4.3 전문 분야

- **의료**: 진단 보조, 의료 기록 요약, 환자 상담.
- **법률**: 판례 검색, 계약서 검토, 법률 자문.
- **교육**: 개인화 튜터, 문제 생성, 과제 채점.
- **금융**: 시장 분석, 리포트 작성, 사기 탐지.

1.4.4 멀티모달

최신 LLM은 텍스트뿐 아니라 이미지, 오디오, 비디오도 처리한다. GPT-4V, Gemini, Claude 3 등은 이미지를 이해하고 설명할 수 있다.

1.5 왜 직접 만들어야 하는가

“HuggingFace에서 모델을 다운로드하면 되는데, 왜 굳이 직접 만들어야 하는가?”라는 질문은 정당하다. 답은 세 가지다.

첫째, 이해의 깊이가 다르다. `model.generate()`를 호출하는 것과, 그 안에서 무슨 일이 일어나는지 아는 것은 차원이 다른 이해다. 직접 만들면 디버깅, 최적화, 커스터마이징이 모두 가능해진다. 추론 속도가 느릴 때 어디가 병목인지 진단할 수 있고, 새로운 아키텍처를 실험할 때 어디를 수정해야

할지 안다.

둘째, 제어력이 생긴다. 추론 속도를 높이고 싶을 때, 메모리를 줄이고 싶을 때, 새로운 아키텍처를 실험하고 싶을 때, 내부를 아는 사람과 모르는 사람의 대응력은 하늘과 땅 차이다. HuggingFace 모델을 그대로 쓰다가 이상한 출력이 나오면? 내부를 아는 사람은 어디를 디버깅해야 할지 안다.

셋째, 미래 대비다. LLM 분야는 빠르게 변한다. 새로운 논문이 매주 쏟아지는데, 그것을 이해하려면 기본기가 탄탄해야 한다. RoPE, GQA, SwiGLU 같은 최신 기법도 결국 어텐션, FFN, 정규화의 변형이다. 이 책을 통해 기본기를 다지면, 2권 Make LLM-advanced에서 다루는 분산 학습, 양자화, 정렬 같은 고급 주제도 자연스럽게 흡수할 수 있다.

이 책의 한계

이 책은 “실제 ChatGPT를 만드는” 것을 목표로 하지 않는다. 1권에서 만드는 모델은 1천만~1억 파라미터 수준의 소규모 GPT로, 품질은 GPT-2 small 정도다. 하지만 그 구조는 GPT-3, GPT-4와 본질적으로 같다. “크기”는 2권에서 다루고, “구조와 원리”를 1권에서 다룬다.

왜 큰 모델을 만들지 않는가? 175B 모델을 학습하려면 수백 개 GPU가 수주일 필요하고, 수억 달러 비용이 든다. 이것은 책으로 따라하기 어렵다. 대신 작은 모델로 원리를 익히면, 큰 모델도 같은 원리라는 것을 알게 된다.

1.6 이 책이 만들 모델

1권을 완독하면 독자는 다음과 같은 모델을 직접 만들게 된다.

- **아키텍처**: GPT 스타일 트랜스포머 디코더 (6~12층, 8헤드, 512~768 임베딩 차원)
- **토큰라이저**: BPE (Byte Pair Encoding)와 Unigram 둘 다 직접 구현
- **학습**: AdamW + cosine LR 스케줄 + warmup
- **생성**: Greedy, Top-K, Top-P, Temperature 샘플링 전부 구현
- **평가**: Cross-entropy loss와 Perplexity 계산

2권 Make LLM-advanced에서는 이 기반 위에 분산 학습(DDP, FSDP, ZeRO), 최신 아키텍처(RoPE, GQA, SwiGLU, MoE), 양자화(INT8/INT4/AWQ), 정렬(SFT, RLHF, DPO, LoRA), 추론 최적화(KV cache, PagedAttention)를 추가한다.

1.7 학습 로드맵

1.8 요약

- 언어 모델은 텍스트 시퀀스에 확률을 할당하는 모델이며, 다음 토큰 예측이 핵심 목표다.
- N-gram에서 RNN/LSTM을 거쳐 2017년 트랜스포머가 등장하며 LLM 시대가 열렸다.
- 스케일링 법칙에 따라 모델, 데이터, 연산을 키우면 성능이 예측 가능하게 향상한다.
- 2022년 이후 정렬(RLHF/DPO)이 “단순 LM”을 “쓸모있는 챗봇”으로 변환시켰다.
- 직접 만들면 이해의 깊이, 제어력, 미래 대비 능력이 모두 확보된다.
- 1권에서는 소규모 GPT를, 2권에서는 ChatGPT 수준의 기법을 다룬다.

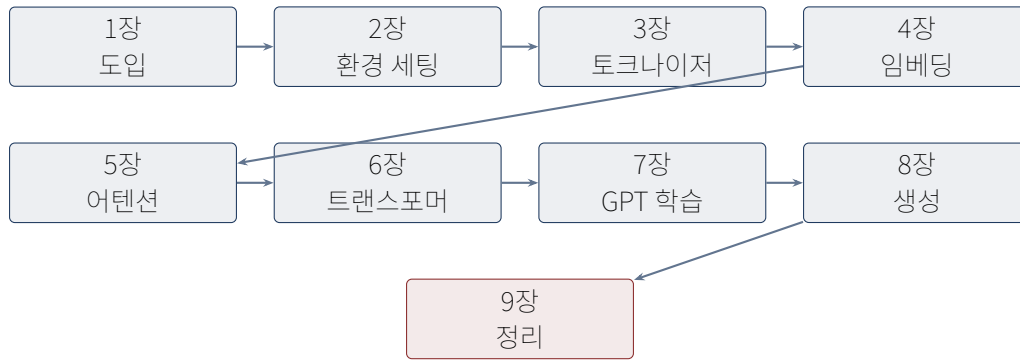


Figure 1.2: 1권 학습 로드맵. 각 장은 이전 장의 모듈을 import하여 사용.

1.9 연습 문제

1. ChatGPT, Claude, Gemini 중 하나를 사용해 다음 질문에 답해보라. “언어 모델이 다음 토큰을 예측하도록 학습된다면, 어떻게 질문에 답할 수 있는가?” 모델의 답변을 평가해보라.
2. 다음 문장의 다음 단어 확률을 사람이라면 어떻게 매길지 생각해보라: “태양은 동쪽에서 ...”. “쬘는”이 99% 이상일 것이다. 왜 이 확률이 높은가?
3. N-gram 모델의 한계 중 “희소성 문제”를 설명하라. 학습하지 않은 3-gram “북극곰이 춤을” 다음 단어를 N-gram은 어떻게 예측할까?
4. 트랜스포머가 RNN보다 병렬 처리에 유리한 이유를 설명하라.
5. 스케일링 법칙에 따르면 모델을 10배 키우면 데이터는 얼마나 키워야 하는가? (Chinchilla 기준)

다음 장에서는 본격적인 구현을 위해 Python, PyTorch, 그리고 이 책의 코드 저장소를 세팅한다.

Chapter 2

개발 환경 세팅

이 장에서는 본격적인 LLM 구현에 앞서 필요한 개발 환경을 세팅한다. Python, PyTorch, 그리고 이 책의 코드 저장소 구조를 살펴보고, 첫 번째 “Hello, LLM” 프로그램을 실행해본다.

2.1 필요한 도구

도구	용도	권장 버전
Python	구현 언어	3.10 이상
PyTorch	딥러닝 프레임워크	2.0 이상
NumPy	수치 계산	1.24 이상
pytest	단위 테스트	7.0 이상
git	버전 관리	최신

GPU는 선택 사항이다. 이 책의 모든 코드는 CPU에서 동작하도록 작성되었다. 다만 대규모 학습을 위해서는 CUDA 지원 GPU가 강력히 권장된다. 2권에서 분산 학습을 다룰 때는 다중 GPU 환경을 가정한다.

2.2 PyTorch 설치

PyTorch는 공식 홈페이지의 설치 가이드를 따른다. CPU 전용 버전과 CUDA 버전 중 자신의 환경에 맞는 것을 선택한다.

```
# CPU 전용 이( 책의 모든 예제는 로CPU 동작)
pip install torch --index-url https://download.pytorch.org/whl/cpu

# CUDA 12.1 지원 (GPU 사용 시)
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

설치 확인은 다음과 같이 한다.

```
1 import torch
2 print(torch.__version__) # 2.x.x
3 print(torch.cuda.is_available()) # GPU 사용 가능 여부
```

2.3 저장소 구조

이 책의 코드는 다음과 같은 구조를 가진다. 각 장에서 다루는 모듈이 별도 파일로 분리되어 있어, 필요한 부분만 가져다 쓸 수 있다.

```

make-llm-basic/
src/make-llm/
  __init__.py
  tokenizer/          # 장3: 토큰나이저
    base.py
    bpe.py
    unigram.py
  model/              # 장4-6: 모델 구조
    embedding.py
    attention.py
    transformer_block.py
    gpt.py
  training/           # 장7: 학습
    dataset.py
    loss.py
    optimizers.py
    trainer.py
  inference/          # 장8: 추론
    sampler.py
    generate.py
  utils/              # 공통 유틸
    config.py
    seed.py
    logging.py
  tests/              # pytest 단위 테스트
  book/               # 이 책의 LaTeX 소스

```

2.4 첫 번째 예제: 텐서 다루기

본격적인 구현에 앞서 PyTorch 텐서의 기본을 복습하자. 텐서는 NumPy 배열과 거의 동일하게 동작하지만, GPU 연산과 자동 미분(`autograd`)을 지원한다는 점이 다르다.

```

1 import torch
2
3 # 텐서 생성
4 x = torch.randn(2, 3)      # 표준정규분포에서 샘플링
5 print(x.shape)             # torch.Size([2, 3])
6 print(x.dtype)             # torch.float32
7
8 # 연산
9 y = x @ x.T                # 행렬곱: [2, 3] @ [3, 2] -> [2, 2]
10 z = x.sum()                # 모든 원소 합 스칼라()
11
12 # 자동 미분
13 w = torch.randn(3, requires_grad=True)
14 loss = (w * w).sum()        # L2 norm^2
15 loss.backward()            # 역전파
16 print(w.grad)              # d(loss)/d(w) = 2*w

```

자동 미분 (Autograd)

PyTorch의 가장 강력한 기능 중 하나. 텐서에 수행하는 모든 연산을 내부적으로 “계산 그래프”로 기록하고, `loss.backward()` 한 번으로 모든 파라미터에 대한 그래디언트를 자동 계산한다. 우리가 직접 미분 식을 유도할 필요가 없다. 이 책의 모든 모델은 이 autograd에 의존한다.

2.5 재현성 확보

딥러닝 실험은 무작위성이 많아 재현성이 중요하다. 시드를 고정하면 같은 코드를 여러 번 실행해도 같은 결과가 나온다.

```
1 from makellm.utils import set_seed
2 set_seed(42) # Python, NumPy, PyTorch 시드를 한 번에 설정
```

이 함수는 Python의 `random`, NumPy의 `numpy.random`, 그리고 PyTorch의 `torch.manual_seed`를 모두 호출한다. CUDA가 존재하면 CUDA 시드도 설정한다.

2.6 단위 테스트 실행

각 장의 코드는 `pytest` 기반 단위 테스트와 함께 제공된다. 코드를 수정한 후에는 반드시 테스트를 실행하여 회귀가 없는지 확인하자.

```
cd make-llm-basic
pytest tests/ -v
```

```
===== 48 passed in 2.47s =====
```

48개 테스트가 모두 통과하면 환경 세팅이 완료된 것이다.

2.7 요약

- PyTorch 2.x와 `pytest`를 설치했다.
- 저장소 구조를 이해했다: `src/make-llm/`에 모듈별로 코드가 분리되어 있다.
- 텐서 연산과 autograd의 기본을 복습했다.
- 재현성을 위해 `set_seed()`를 사용한다.
- `pytest tests/`로 단위 테스트를 실행할 수 있다.

다음 장에서는 본격적으로 토큰라이저를 구현한다. 텍스트를 숫자로 바꾸는 이 첫 번째 변환이 모든 LLM의 출발점이다.

Chapter 3

토큰나이저 만들기

이 장에서는 텍스트를 숫자 시퀀스로 변환하는 **토큰나이저(tokenizer)**를 직접 구현한다. 두 가지 주요 알고리즘인 BPE(Byte Pair Encoding)와 Unigram을 다루며, 각각의 학습, 인코딩, 디코딩 과정을 코드로 살펴본다.

3.1 왜 토큰나이저가 필요한가

신경망은 숫자만 이해한다. “Hello, world!”라는 문자열을 그대로 모델에 넣을 수 없다. 텍스트를 정수 시퀀스로 변환하는 첫 단계가 필요한데, 이를 **토큰나이저**가 담당한다. 가장 단순한 방법은 “글자 단위 토큰나이저”로, 모든 문자를 별도의 토큰으로 취급하는 것이다. 하지만 이는 시퀀스가 너무 길어지고 단어의 의미를 잡기 어렵다.

반대 극단은 “단어 단위 토큰나이저”로, 공백으로 분리된 단어를 하나의 토큰으로 취급한다. 하지만 이는 어휘 크기가 폭발하고(영어만 해도 수십만 단어), 학습하지 않은 단어(out-of-vocabulary, OOV)를 처리하지 못한다.

현대 LLM은 이 두 극단의 중간인 **서브워드(subword)** 토큰나이저를 사용한다. 자주 등장하는 단어는 그대로 하나의 토큰으로, 희귀한 단어는 더 작은 서브워드로 분할한다. “unhappiness”는 “un” + “happiness” 또는 “un” + “happi” + “ness”로 쪼개질 수 있다. 이 방식은 어휘 크기를 적절히 유지하면서 OOV 문제를 해결한다.

3.1.1 세 가지 접근 비교

방식	어휘 크기	OOV 처리	시퀀스 길이
글자 단위	작음 (~100)	없음	매우 김
단어 단위	매우 큼 (~100K)	처리 못함	짧음
서브워드	중간 (~10K~50K)	부분단어로 분해	중간

3.2 특수 토큰

모든 토큰나이저는 몇 가지 **특수 토큰(special token)**을 가진다. 우리의 구현에서는 네 가지를 사용한다.

토큰	ID	용도
<pad>	0	배치 내 시퀀스 길이를 맞추기 위한 패딩
<bos>	1	시퀀스의 시작을 표시 (Beginning Of Sequence)
<eos>	2	시퀀스의 끝을 표시 (End Of Sequence)
<unk>	3	어휘에 없는 토큰 (Unknown)

이 토큰들은 인덱스가 고정되어 있어 모델이 특별히 취급할 수 있다. 예를 들어 생성 시 <eos>가 나오면 추론을 멈추고, <pad> 위치는 loss 계산에서 무시한다.

특수 토큰 ID 고정의 이유

특수 토큰의 ID를 0~3으로 고정하면 모델 코드가 단순해진다. “if token_id == 0: mask_out” 같은 하드코딩이 가능하다. 또한 체크포인트 호환성이 보장된다: 토큰나이저를 다시 학습해도 특수 토큰 ID는 항상 같으므로 모델 가중치를 그대로 재사용할 수 있다.

3.3 베이스 클래스

모든 토큰나이저가 따라야 할 인터페이스를 추상 클래스로 정의한다.

```

1 from abc import ABC, abstractmethod
2 from dataclasses import dataclass
3
4 @dataclass
5 class SpecialTokens:
6     pad: str = "<pad>" # ID 0
7     bos: str = "<bos>" # ID 1
8     eos: str = "<eos>" # ID 2
9     unk: str = "<unk>" # ID 3
10
11     @property
12     def tokens(self) -> list[str]:
13         return [self.pad, self.bos, self.eos, self.unk]
14
15     @property
16     def pad_id(self) -> int: return 0
17     @property
18     def bos_id(self) -> int: return 1
19     @property
20     def eos_id(self) -> int: return 2
21     @property
22     def unk_id(self) -> int: return 3
23
24
25 class Tokenizer(ABC):
26     def __init__(self, special_tokens: SpecialTokens | None = None):
27         self.special = special_tokens or SpecialTokens()
28         self.id_to_token: list[str] = list(self.special.tokens)
29         self.token_to_id: dict[str, int] = {t: i for i, t in enumerate(self.id_to_token)}
30
31     @abstractmethod
32     def _train(self, corpus: list[str], vocab_size: int) -> None:
33         """ 어휘 학습 자식(클래스에서 구현) """
34

```

```

35 @abstractmethod
36 def _tokenize(self, text: str) -> list[str]:
37     """텍스트를 토큰 리스트로 분할 자식(클래스에서 구현)"""
38
39 def encode(self, text: str, add_bos=False, add_eos=False) -> list[int]:
40     tokens = self._tokenize(text)
41     ids = [self.token_to_id.get(t, self.special.unk_id) for t in tokens]
42     if add_bos: ids = [self.special.bos_id] + ids
43     if add_eos: ids = ids + [self.special.eos_id]
44     return ids
45
46 def decode(self, ids: list[int]) -> str:
47     tokens = [self.id_to_token[i] for i in ids
48               if self.id_to_token[i] not in self.special.tokens]
49     raw = self._tokens_to_bytes(tokens)
50     return self._postprocess(raw.decode("utf-8", errors="replace"))

```

추상 클래스의 역할

Tokenizer는 “인터페이스”를 정의한다. 자식 클래스(BPE, Unigram)는 `_train`과 `_tokenize`를 구현해야 한다. 이렇게 하면 학습 알고리즘은 달라도 `encode/decode` API는 동일하게 유지되어, 상위 코드(데이터셋, 트레이너)가 토크나이저 종류에 무관하게 동작한다. 이것이 “다형성(polymorphism)”의 가치다. 트레이너는 `tokenizer.encode(text)`만 호출하면 되고, 내부적으로 BPE가 돌든 Unigram이 돌든 신경 쓰지 않는다. 나중에 “Sentence-Piece”나 “WordPiece” 같은 새 알고리즘을 추가해도 트레이너 코드는 수정할 필요가 없다.

3.4 BPE 토크나이저

BPE(Byte Pair Encoding)는 1994년 데이터 압축을 위해 제안된 알고리즘을 2016년 Sennrich et al.이 신경망 기계번역에 도입한 것이다. GPT-2, GPT-3, LLaMA 등이 사용한다.

3.4.1 알고리즘 직관

BPE의 핵심 아이디어는 단순하다:

1. 텍스트를 바이트 단위로 분해한다 (모든 문자를 표현 가능, OOV 없음).
2. 인접한 바이트 쌍 중 가장 자주 등장하는 쌍을 찾는다.
3. 그 쌍을 새로운 토큰으로 병합한다.
4. 목표 어휘 크기에 도달할 때까지 2~3을 반복한다.

처음에는 모든 텍스트가 “바이트의 나열”이다. 예를 들어 “lower”는 [l, o, w, e, r]로 표현된다. 학습을 진행하면서:

- “lo”가 자주 등장하면 병합: [lo, w, e, r]
- “er”가 자주 등장하면 병합: [lo, w, er]
- “low”가 자주 등장하면 병합: [low, er]
- “lower”가 자주 등장하면 병합: [lower]

최종적으로 “lower”가 하나의 토큰이 되지만, 희귀한 단어는 부분단어로 남는다. “lowering”은 “lower” + “ing”이 될 수 있다.

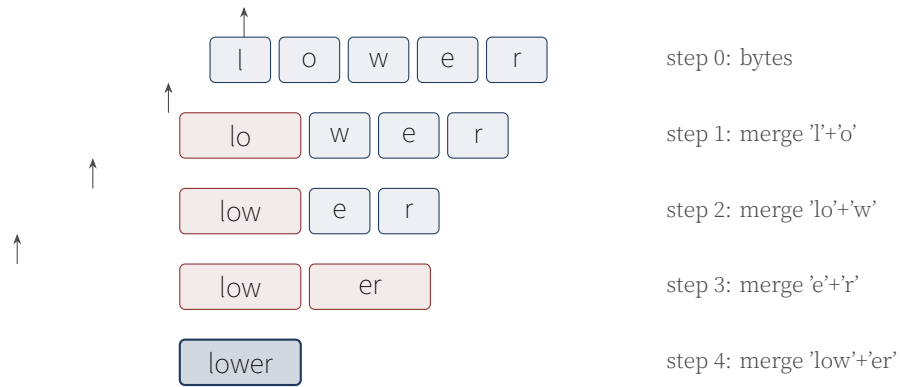


Figure 3.1: BPE 학습 과정. “lower”가 바이트에서 시작해 점진적으로 병합되어 하나의 토큰이 되는 과정.

3.4.2 왜 바이트 단위인가?

GPT-2부터 BPE는 “바이트 단위”로 동작한다. 이전에는 문자(character) 단위를 썼는데, 문제가 있었다:

- 이모지, 희귀 한자 등 어휘에 없는 문자는 <unk> 처리됨.
- 다국어 코퍼스에서 문자 집합이 너무 커짐.

바이트 단위면 모든 UTF-8 문자를 1~4바이트로 표현할 수 있고, 어휘의 “바이트” 부분은 항상 256개로 고정된다. OOV가 사실상 사라진다.

3.4.3 구현: 학습

```

1 from collections import Counter
2 from .base import Tokenizer
3
4 class BPETokenizer(Tokenizer):
5     def __init__(self, special_tokens=None):
6         super().__init__(special_tokens=special_tokens)
7         self.merges: list[tuple[str, str]] = []
8         self._merge_rank: dict[tuple[str, str], int] = {}
9
10    def _train(self, corpus: list[str], vocab_size: int) -> None:
11        # 1. 특수 토큰 + 개256 바이트로 초기 어휘 구성
12        self.id_to_token = list(self.special_tokens)
13        for b in range(256):
14            self.id_to_token.append(chr(b))
15        self.token_to_id = {t: i for i, t in enumerate(self.id_to_token)}
16
17        # 2. 코퍼스를 단어 단위로 분할
18        word_freqs = Counter()
19        for text in corpus:
20            for word in self._pre_tokenize(text):
21                word_freqs[word] += 1
22
23        # 3. 각 단어를 바이트 토큰 리스트로 변환
24        word_to_tokens = {
25            w: [chr(b) for b in w.encode("utf-8")]
26            for w in word_freqs
27        }
28

```

```

29 # 4. 병합 루프
30 self.merges = []
31 target = vocab_size - len(self.id_to_token)
32 for _ in range(max(target, 0)):
33     pair_freqs = Counter()
34     for word, freq in word_freqs.items():
35         toks = word_to_tokens[word]
36         for i in range(len(toks) - 1):
37             pair_freqs[(toks[i], toks[i+1])] += freq
38     if not pair_freqs:
39         break
40     best = max(pair_freqs.items(), key=lambda x: (x[1], x[0]))
41     if best[1] < 2:
42         break
43     new_token = best[0][0] + best[0][1]
44     self.merges.append(best[0])
45     self.id_to_token.append(new_token)
46     self.token_to_id[new_token] = len(self.id_to_token) - 1
47     for word in word_to_tokens:
48         word_to_tokens[word] = self._merge_in_list(
49             word_to_tokens[word], best[0], new_token
50         )
51
52     self._merge_rank = {p: i for i, p in enumerate(self.merges)}

```

병합 우선순위 안정성

`max(pair_freqs.items(), key=lambda x: (x[1], x[0]))`에서 `(x[1], x[0])` 튜플로 정렬하는 이유는 동점 처리 때문이다. 빈도가 같은 쌍이 여러 개일 때, 사전순으로 가장 앞선 쌍을 선택하여 결과를 결정론적으로 만든다. 이렇게 해야 같은 코퍼스로 학습하면 항상 같은 병합 순서가 나온다.

3.4.4 인코딩

학습된 병합 규칙을 새 텍스트에 적용하려면, 가장 우선순위가 높은(먼저 학습된) 병합부터 적용한다.

```

1 def _tokenize(self, text: str) -> list[str]:
2     tokens = []
3     for word in self._pre_tokenize(text):
4         bs = word.encode("utf-8")
5         word_tokens = [chr(b) for b in bs]
6         word_tokens = self._apply_merges(word_tokens)
7         tokens.extend(word_tokens)
8     return tokens
9
10 def _apply_merges(self, tokens: list[str]) -> list[str]:
11     """학습된 병합 규칙을 우선순위 순으로 적용."""
12     while len(tokens) >= 2:
13         # 가장 가rank 낮은 우선순위( 높은) 쌍 찾기
14         best_idx, best_rank = -1, len(self.merges) + 1
15         for i in range(len(tokens) - 1):
16             rank = self._merge_rank.get((tokens[i], tokens[i+1]), -1)
17             if 0 <= rank < best_rank:
18                 best_rank, best_idx = rank, i
19         if best_idx < 0:
20             break

```

```

21 merged = tokens[best_idx] + tokens[best_idx + 1]
22 tokens = tokens[:best_idx] + [merged] + tokens[best_idx + 2:]
23 return tokens

```

3.4.5 전처리: 공백 처리

BPE는 공백을 특별히 처리해야 단어 경계를 보존할 수 있다. GPT-2는 공백을 특수 문자 G(U+0120)로 치환한다. 우리는 SentencePiece 방식을 따라 U+2581(▯)로 치환한다.

```

1 def _pre_tokenize(self, text: str) -> list[str]:
2     text = text.replace(" ", "▯")
3     text = text.replace("\n", "▯").replace("\t", "▯")
4     while "▯▯" in text:
5         text = text.replace("▯▯", "▯")
6     if not text.startswith("▯"):
7         text = "▯" + text
8     parts = text.split("▯")
9     return ["▯" + p for i, p in enumerate(parts) if i > 0 and p]

```

공백 치환의 함정

“Hello world”를 “▯Hello▯world”로 치환한 후 split하면 [“▯Hello”, “▯world”]가 된다. 단어마다 ▯이 앞에 붙어 있어, 디코딩 시 ▯을 다시 공백으로 바꾸면 원래 문장이 복원된다. 하지만 “Hello, world!”처럼 구두점이 있으면 주의해야 한다. 단순 split은 “▯Hello,”를 만들어 쉼표가 단어의 일부가 되어버린다. 실제 GPT-2는 정규식 패턴으로 단어/구두점을 분리하지만, 이 책에서는 단순화를 위해 생략한다.

3.4.6 디코딩: 바이트 복원

BPE의 까다로운 부분은 디코딩이다. 토큰이 chr(byte)들의 시퀀스이므로, 각 문자의 코드포인트를 바이트로 취급하여 원래 바이트 스트림을 복원한 뒤 UTF-8로 디코딩해야 한다.

```

1 def _tokens_to_bytes(self, tokens: list[str]) -> bytes:
2     """BPE 토큰을 바이트로 변환. 각 문자의 코드포인트를 바이트로."""
3     out = bytearray()
4     for tok in tokens:
5         for c in tok:
6             out.append(ord(c) & 0xFF)
7     return bytes(out)

```

예를 들어 “▯Hello” 토큰이 있으면, ▯(U+2581)은 UTF-8로 0xE2 0x96 0x81이므로 3바이트이지만, BPE 학습 시 “▯” 문자 자체를 3개의 개별 바이트 토큰으로 처리했으므로 chr(0xE2) + chr(0x96) + chr(0x81)로 표현된다. 디코딩 시 각 chr(b)에서 ord()로 바이트를 추출하면 원래 바이트 스트림 0xE2 0x96 0x81 0x48 0x65 0x6C 0x6C 0x6F를 얻고, 이를 UTF-8로 디코딩하면 “▯Hello”가 된다.

3.5 Unigram 토큰나이저

Unigram 토큰나이저는 Kudo (2018)가 SentencePiece의 일부로 제안한 방식이다. BPE가 “빈도 기반 병합”이라면, Unigram은 “확률 기반 분할”이다.

3.5.1 직관

Unigram은 각 토큰에 확률을 할당한다. 문장 “lower”에 대해 가능한 분할은 여러 가지다:

- [l, o, w, e, r]: 5개 토큰
- [lo, w, er]: 3개 토큰
- [low, er]: 2개 토큰
- [lower]: 1개 토큰

각 분할의 확률은 토큰 확률의 곱으로 계산된다. 가장 높은 확률을 갖는 분할을 선택한다. “lower”가 어휘에 있고 확률이 높으면 [lower]가 선택되지만, 없으면 더 작은 단위로 분할된다.

3.5.2 알고리즘

1. 코퍼스에서 모든 부분문자열을 후보 토큰으로 추출하고 빈도를 센다.
2. 빈도를 확률로 변환하여 각 토큰에 로그 확률 점수를 할당.
3. EM(Expectation-Maximization) 반복:
 - E-step: Viterbi 알고리즘으로 각 단어의 최적 분할 탐색.
 - M-step: 분할 결과로 토큰 확률 갱신.
4. 가장 점수가 낮은 토큰을 제거하며 어휘 축소.

3.5.3 Viterbi 디코딩

주어진 단어에 대해 “가장 높은 총 로그 확률을 갖는 토큰 분할”을 동적 계획법으로 찾는다.

```
1 def _viterbi(self, word: str) -> tuple[list[str], float]:
2     n = len(word)
3     # dp[i] = 최대( 점수, 이전 인덱스, 이전 토큰)
4     dp = [(-float("inf"), -1, "") for _ in range(n + 1)]
5     dp[0] = (0.0, -1, "")
6     for i in range(1, n + 1):
7         for j in range(max(0, i - 16), i):
8             substr = word[j:i]
9             score = self.token_scores.get(substr, -20.0)
10            cand = dp[j][0] + score
11            if cand > dp[i][0]:
12                dp[i] = (cand, j, substr)
13
14     # 역추적
15     path, i = [], n
16     while i > 0:
17         _, j, tok = dp[i]
18         path.append(tok)
19         i = j
20     path.reverse()
21     return path, dp[n][0]
```

Viterbi의 복잡도

$O(n^2)$ 대신 $O(n \cdot L)$ 로 줄인 비결은 $\max(0, i-16)$ 부분이다. 토큰 길이가 16을 넘는 경우가 드물기 때문에, j 의 범위를 제한하여 탐색 공간을 줄인다. 실제 SentencePiece도 비슷한 최적화를

사용한다.

3.6 BPE vs Unigram 비교

	BPE	Unigram
학습 방식	빈도 기반 병합	확률 기반 분할 (EM)
인코딩	병합 규칙 순서 적용	Viterbi로 최적 분할
결정론적	O (같은 입력, 같은 출력)	O
다중 분할 지원	X	O (샘플링 가능)
주요 사용처	GPT-2, GPT-3, LLaMA	T5, ALBERT, mBART

3.7 토큰라이저 테스트

구현이 끝났으면 단위 테스트로 검증한다. 가장 중요한 테스트는 **라운드트립(round-trip) 테스트**: 인코딩 후 디코딩하면 원문이 복원되어야 한다는 것이다.

```
1 def test_encode_decode_roundtrip():
2     tok = BPETokenizer()
3     tok.train(CORPUS, vocab_size=200)
4     text = "the quick fox"
5     ids = tok.encode(text)
6     decoded = tok.decode(ids)
7     assert " ".join(decoded.split()) == "the quick fox"
8
9 def test_bos_eos_addition():
10    tok = BPETokenizer()
11    tok.train(CORPUS, vocab_size=100)
12    ids = tok.encode("hello", add_bos=True, add_eos=True)
13    assert ids[0] == tok.special.bos_id
14    assert ids[-1] == tok.special.eos_id
15
16 def test_unknown_token_handled():
17     """어휘에 없는 문자도 바이트 단위로 처리되어 가unk 발생하지 않아야 함."""
18     tok = BPETokenizer()
19     tok.train(CORPUS, vocab_size=100)
20     ids = tok.encode("안녕")
21     assert tok.special.unk_id not in ids # 바이트로 분해되므로 unk 없음
22
23 def test_save_load(tmp_path):
24     tok = BPETokenizer()
25     tok.train(CORPUS, vocab_size=150)
26     path = tmp_path / "tok.json"
27     tok.save(path)
28     tok2 = BPETokenizer.load(path)
29     text = "the quick fox"
30     assert tok.encode(text) == tok2.encode(text)
```

테스트 철학

이 책의 모든 모듈은 세 가지 수준의 테스트를 갖는다:

- **형상 테스트(shape test)**: 텐서의 shape이 예상과 일치하는지.
- **수치 테스트(numeric test)**: 작은 입력에 대해 손계산 결과와 비교.
- **통합 테스트**: 여러 모듈을 연결했을 때 end-to-end로 동작.

이 세 가지를 통과한 코드만 책에 수록된다.

라운드트립 테스트는 통합 테스트의 한 예다. “인코딩 후 디코딩하면 원문이 나온다”는 자명해 보이지만, 실제로는 공백 처리, 멀티바이트 문자, 특수 토큰 제거 등 여러 단계에서 버그가 생길 수 있다. 이 테스트가 통과하면 전체 파이프라인이 올바르게 동작함을 확신할 수 있다.

3.8 실습: 실제 말뭉치로 토큰나이저 학습

책의 scripts/tiny_train.py 데모를 실행하면 실제로 BPE 토큰나이저를 학습하는 과정을 볼 수 있다.

```
cd make-llm-basic
python scripts/tiny_train.py --vocab 500
```

출력 예:

```
1 [1/5] 토큰나이저 학습 중...
2   완료: 개308 토큰, 초1.2
3   샘플: 'the quick brown fox jumps over the lazy dog'
4   → 개18 토큰: [264, 282, 291, 267, 261, 110, ...]
```

3.9 요약

- 토큰나이저는 텍스트를 정수 시퀀스로 변환하는 LLM의 첫 단계다.
- 서브워드 방식이 어휘 크기와 OOV 사이의 균형을 잡는다.
- BPE는 빈도 기반 병합, Unigram은 확률 기반 분할.
- 특수 토큰 <pad>/<bos>/<eos>/<unk>는 ID 0~3에 고정.
- 바이트 단위 BPE는 OOV를 사실상 제거한다.
- Viterbi 알고리즘으로 Unigram의 최적 분할을 $O(n)$ 에 찾는다.
- 모든 구현은 pytest 단위 테스트로 검증된다.

3.10 연습 문제

1. BPE 학습 시 “가장 빈도가 높은 쌍” 대신 “가장 빈도가 낮은 쌍”을 병합하면 어떻게 될까? 결과를 예측하고 실험해보라.
2. Unigram 토큰나이저에서 score = -20.0 패널티 값을 -5.0으로 바꾸면 어떤 일이 일어나는가?
3. 한국어 코퍼스 “안녕하세요 만나서 반갑습니다”를 BPE로 학습한다고 하자. 어휘 크기가 100일 때와 1000일 때 어떤 차이가 나는가?
4. Viterbi에서 토큰 길이 상한을 16이 아니라 4로 줄이면 어떤 토큰이 사라지는가?

5. <bos> 토큰이 왜 필요한지 생각해보라. 없어도 모델이 동작할까?
다음 장에서는 토큰 ID를 밀집 벡터로 변환하는 임베딩 레이어를 구현한다.

Chapter 4

임베딩과 포지셔널 인코딩

이 장에서는 토큰 ID를 밀집 벡터(dense vector)로 변환하는 임베딩(embedding)과 시퀀스의 순서 정보를 모델에 주입하는 포지셔널 인코딩(positional encoding)을 구현한다.

4.1 왜 임베딩이 필요한가

3장에서 토큰라이저가 “Hello”를 정수 ID 247로 바꿔주었다. 하지만 이 정수를 그대로 신경망에 넣으면 문제가 생긴다. 첫째, 정수 247과 248이 “비슷한 단어”라는 보장이 없다. “cat”이 247, “dog”이 248일 수도 있고 “zebra”가 248일 수도 있다. 둘째, 정수를 곱하거나 더하는 연산이 의미를 갖지 않는다. “cat + 1 = dog”이 성립하지 않는다.

해결책은 각 토큰 ID를 밀집 벡터로 변환하는 것이다. 어휘 크기가 V 이고 임베딩 차원이 d 일 때, 임베딩은 $V \times d$ 크기의 학습 가능한 행렬 E 다. 토큰 ID i 가 들어오면 E 의 i 번째 행을 가져온다. 이 행벡터가 그 토큰의 “의미적 표현”이 된다.

왜 “밀집”인가?

원-핫 인코딩(one-hot)은 어휘 크기 V 의 희소 벡터로 토큰을 표현한다. 예를 들어 1만 어휘에 대해 ID 247은 246개의 0과 1개의 1, 나머지 0으로 이루어진 벡터다. 이를 E 와 곱하면 결국 E 의 247번째 행을 가져오는 것과 같다. 임베딩은 이 “원-핫 $\times$ 행렬”을 lookup 연산 하나로 수행하는 효율적 구현이다.

학습이 진행되면 비슷한 의미의 단어가 벡터 공간에서 가까워진다. “cat”과 “dog”은 가깝고, “cat”과 “car”는 멀다. 이것이 임베딩의 마법이다: 학습 전에는 무작위 벡터였지만, 학습 후에는 의미를 담은 좌표가 된다.

4.2 임베딩의 수학적 정의

임베딩 행렬 $E \in \mathbb{R}^{V \times d}$ 가 있을 때, 토큰 ID i 의 임베딩은 $E[i] \in \mathbb{R}^d$ 다. 이것은 단순히 E 의 i 번째 행을 선택하는 연산이며, 미분 가능하다 (backward 시 선택된 행만 그래디언트를 받는다).

배치 입력 $\mathbf{x} \in \mathbb{Z}^{B \times L}$ (배치 B , 시퀀스 길이 L)에 대해 임베딩 출력은 $\mathbf{X} \in \mathbb{R}^{B \times L \times d}$ 가 된다.

4.3 Token Embedding 구현

PyTorch의 `nn.Embedding`은 정확히 이 lookup 연산을 수행한다. 우리는 이를 얇게 래핑하여 초기화 로직을 추가한다.

```
1 import math
2 import torch.nn as nn
3
```



```

4 class TokenEmbedding(nn.Module):
5     def __init__(self, vocab_size: int, d_model: int, pad_id: int = 0):
6         super().__init__()
7         self.vocab_size = vocab_size
8         self.d_model = d_model
9         self.pad_id = pad_id
10        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=pad_id)
11        self._init_weights()
12
13    def _init_weights(self):
14        # 정규분포  $N(0, 1/\sqrt{d_{\text{model}}})$  초기화 (GPT-2 스타일)
15        nn.init.normal_(self.embedding.weight, mean=0.0,
16                        std=1.0 / math.sqrt(self.d_model))
17        # 패딩 토큰은 0으로
18        with torch.no_grad():
19            self.embedding.weight[self.pad_id].fill_(0.0)
20
21    def forward(self, token_ids: torch.Tensor) -> torch.Tensor:
22        # [batch, seq] -> [batch, seq, d_model]
23        return self.embedding(token_ids)

```

초기화의 중요성

딥러닝에서 가중치 초기화는 학습 성패를 좌우한다. 너무 크면 활성화값이 폭발하고, 너무 작으면 그래디언트가 사라진다. GPT-2는 표준편차 $1/\sqrt{d_{\text{model}}}$ 의 정규분포를 사용하며, 이는 초기 임베딩 벡터의 norm이 대략 1이 되도록 하는 합리적 선택이다.

왜 하필 $1/\sqrt{d}$ 인가? d 차원 벡터의 각 원소가 $\mathcal{N}(0, \sigma^2)$ 일 때, 벡터 norm의 기댓값은 $\sqrt{d} \cdot \sigma$ 다. $\sigma = 1/\sqrt{d}$ 면 norm이 1 근처가 된다. 이렇게 하면 임베딩 벡터의 스케일이 레이어 출력과 비슷하여 학습이 안정된다.

4.3.1 padding_idx의 역할

`nn.Embedding`의 `padding_idx` 매개변수는 패딩 토큰의 임베딩을 항상 0으로 유지한다. 이렇게 하면:

- 패딩 위치의 출력이 0이 되어 어텐션 계산에 영향을 주지 않는다.
- backward 시 패딩 행의 그래디언트가 0이 되어 가중치가 갱신되지 않는다.

4.4 포지셔널 인코딩

트랜스포머는 어텐션만으로 시퀀스를 처리하는데, 어텐션 자체는 “순서”라는 개념이 없다. 입력을 섞어도 출력은 (재배열된 채로) 같다. 이를 **순서 불변성(permutation invariance)**이라고 한다. 자연어에서는 순서가 의미를 결정하므로(“개가 사람을 물었다” vs “사람이 개를 물었다”), 순서 정보를 명시적으로 주입해야 한다.

4.4.1 직관: 왜 순서가 중요한가?

어텐션은 “각 토큰이 다른 토큰에 얼마나 주의를 기울일지”를 학습한다. 하지만 “첫 번째 토큰”과 “마지막 토큰”의 구분이 없다. 문장 “나는 밥을 먹었다”에서 “먹었다”가 주어 “나는”과 연결되어야 하는데, 어텐션은 “5번째 토큰이 1번째 토큰을 본다”는 정보 없이는 이 연결을 학습하기 어렵다.

포지셔널 인코딩은 각 위치에 고유한 “위치 벡터”를 더해 주어 모델이 위치를 구분할 수 있게 한다.

4.4.2 Sinusoidal 위치 인코딩 (Vaswani et al. 2017)

원본 트랜스포머 논문이 제안한 방식. 사인과 코사인 함수로 위치를 인코딩한다.

Sinusoidal Positional Encoding

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

여기서 pos 는 위치, i 는 차원 인덱스. 짝수 차원은 sin, 홀수 차원은 cos.

이 방식의 장점은 학습 가능한 파라미터가 없다는 것이다. 또한 상대적 위치에 대한 선형 관계가 성립하여 모델이 쉽게 학습할 수 있다. $\sin(\alpha + \beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta$ 이므로 위치 $pos + k$ 의 인코딩은 위치 pos 의 인코딩의 선형 변환으로 표현된다.

```

1 class PositionalEncoding(nn.Module):
2     def __init__(self, d_model: int, max_len: int = 512, mode: str = "learned"):
3         super().__init__()
4         self.mode = mode
5         if mode == "sinusoidal":
6             pe = torch.zeros(max_len, d_model)
7             position = torch.arange(0, max_len).unsqueeze(1).float()
8             div_term = torch.exp(
9                 torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
10            )
11            pe[:, 0::2] = torch.sin(position * div_term)
12            pe[:, 1::2] = torch.cos(position * div_term)
13            self.register_buffer("pe", pe.unsqueeze(0)) # [1, max_len, d_model]
14        else: # learned
15            self.pe = nn.Embedding(max_len, d_model)
16            nn.init.normal_(self.pe.weight, mean=0.0, std=1.0/math.sqrt(d_model))
17
18        def forward(self, seq_len: int) -> torch.Tensor:
19            # [1, seq_len, d_model] 반환
20            if seq_len > self.max_len:
21                raise ValueError(f"Sequence length {seq_len} exceeds max_len {self.max_len}")
22            if self.mode == "sinusoidal":
23                return self.pe[:, :seq_len, :]
24            else:
25                positions = torch.arange(seq_len, device=self.pe.weight.device)
26                return self.pe(positions).unsqueeze(0)

```

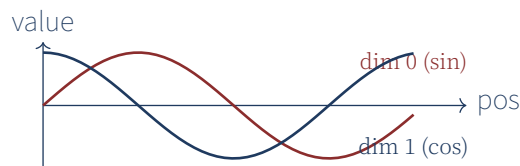


Figure 4.1: Sinusoidal 위치 인코딩의 처음 두 차원. sin과 cos이 위치에 따라 주기적으로 변한다.

4.4.3 학습 가능한 위치 임베딩 (GPT-2 스타일)

GPT-2, GPT-3 등은 위치마다 학습 가능한 임베딩 벡터를 사용한다. 이 방식은 충분한 데이터가 있을 때 더 유연하지만, 학습 시 본 적 없는 길이(예: 학습 시 1024였는데 추론 시 2048)에는 대응하기 어렵다.

Sinusoidal vs Learned

- **Sinusoidal**: 파라미터 없음, extrapolation 가능 (학습 시 본 길이보다 긴 시퀀스에도 대응). 하지만 실제 성능은 학습 가능 방식보다 약간 떨어지는 경향.
- **Learned**: 성능 약간 우수, 하지만 고정 길이. extrapolation 어려움.

최신 모델들은 RoPE(Rotary Position Embedding)를 사용하여 두 방식의 장점을 결합한다. 2권 5장에서 다룬다.

4.5 임베딩 합성

토큰 임베딩과 위치 임베딩을 더해서 최종 입력 표현을 만든다. Vaswani et al.은 임베딩에 $\sqrt{d_{model}}$ 을 곱하여 스케일을 맞추는 것을 제안했는데, 이는 위치 임베딩과 스케일을 비슷하게 유지하기 위함이다.

```
1 class EmbeddingStack(nn.Module):
2     def __init__(self, vocab_size, d_model, max_len=512,
3                 pad_id=0, pos_mode="learned", dropout=0.1):
4         super().__init__()
5         self.token_emb = TokenEmbedding(vocab_size, d_model, pad_id=pad_id)
6         self.pos_emb = PositionalEmbedding(d_model, max_len, mode=pos_mode)
7         self.dropout = nn.Dropout(dropout)
8         self.scale = math.sqrt(d_model)
9
10    def forward(self, token_ids: torch.Tensor) -> torch.Tensor:
11        batch, seq = token_ids.shape
12        tok = self.token_emb(token_ids) * self.scale # 스케일링
13        pos = self.pos_emb(seq)
14        return self.dropout(tok + pos)
```

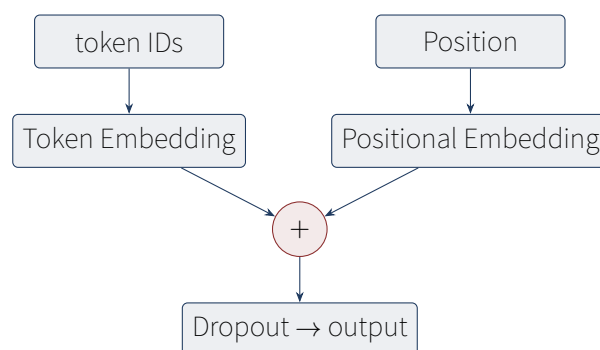


Figure 4.2: EmbeddingStack 구조. 토큰 임베딩과 위치 임베딩을 더한 후 드롭아웃을 적용한다.

4.5.1 왜 스케일링이 필요한가?

토큰 임베딩은 $\mathcal{N}(0, 1/d)$ 로 초기화되어 있어 원소의 분산이 $1/d$ 다. 위치 임베딩 (sinusoidal)의 원소는 $[-1, 1]$ 범위로 분산이 약 $1/2$ 이다. 스케일링 없이 더하면 위치 정보가 토큰 정보를 압도한다. $\sqrt{d}$ 를 곱하면 토큰 임베딩의 분산이 1이 되어 위치 임베딩과 균형을 이룬다.

4.6 Layer Normalization 소개

트랜스포머에서 학습을 안정화하기 위해 레이어 정규화(Layer Normalization)를 사용한다. 다음 장의 어텐션에서도 등장하므로 여기서 소개한다.

Layer Normalization

$$\text{LN}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

여기서 μ, σ^2 은 마지막 차원을 따라 계산한 평균과 분산. γ, β 는 학습 가능한 파라미터.

PyTorch에서는 `nn.LayerNorm(d_model)` 한 줄로 사용한다. 2권에서는 LayerNorm의 변형인 RMSNorm도 다룬다.

LayerNorm vs BatchNorm

BatchNorm은 배치 차원을 따라 정규화하여 학습과 추론 시 통계가 다른 문제가 있다. LayerNorm은 각 샘플 내부에서 정규화하여 배치 크기에 무관하게 동작한다. 이것이 LLM에서 LayerNorm을 쓰는 이유다: 배치 크기 1인 추론에서도 안정적으로 동작한다.

4.7 테스트

임베딩 모듈의 핵심 테스트는 출력 shape 확인과 그래디언트 흐름 확인이다.

```
1 def test_token_embedding_shape():
2     emb = TokenEmbedding(vocab_size=100, d_model=32)
3     ids = torch.randint(0, 100, (2, 8)) # [batch=2, seq=8]
4     out = emb(ids)
5     assert out.shape == (2, 8, 32)
6
7 def test_positional_sinusoidal():
8     pe = PositionalEmbedding(d_model=32, max_len=64, mode="sinusoidal")
9     out = pe(seq_len=16)
10    assert out.shape == (1, 16, 32)
11
12 def test_embedding_stack():
13    stack = EmbeddingStack(vocab_size=100, d_model=32, max_len=64, dropout=0.0)
14    ids = torch.randint(0, 100, (2, 8))
15    out = stack(ids)
16    assert out.shape == (2, 8, 32)
17
18 def test_positional_exceeds_max_len_raises():
19    pe = PositionalEmbedding(d_model=32, max_len=32, mode="learned")
20    with pytest.raises(ValueError):
21        pe(seq_len=64) # max_len 초과
```

4.8 실습: 임베딩 시각화

`scripts/tiny_train.py`에서 학습된 임베딩을 PCA로 2차원에 투영하면 단어들이 어떻게 분포하는지 볼 수 있다. 충분한 학습 후에는 비슷한 단어들이 가깝게 모이는 것을 확인할 수 있다.

4.9 요약

- 임베딩은 토큰 ID를 밀집 벡터로 변환하는 학습 가능한 lookup 테이블이다.
- 초기화는 $N(0, 1/\sqrt{d_{model}})$ 을 사용하고, 패딩 토큰은 0으로 설정한다.
- 포지셔널 인코딩은 트랜스포머에 순서 정보를 주입한다. sinusoidal과 learned 두 방식이 있다.
- 토큰 임베딩과 위치 임베딩을 더하고 드롭아웃을 적용하여 최종 입력 표현을 만든다.
- $\sqrt{d_{model}}$ 스케일링으로 두 임베딩의 분산을 균형 맞춘다.
- LayerNorm은 배치 크기에 무관한 정규화로 LLM 학습 안정화에 필수.

4.10 연습 문제

1. 임베딩 차원 d 가 너무 작으면 (예: 8) 어떤 문제가 생기는가? 너무 크면 (예: 4096) 어떤 문제가 생기는가?
2. Sinusoidal 위치 인코딩에서 10000이라는 상수를 100으로 바꾸면 어떤 일이 일어나는가?
3. 토큰 임베딩에 $\sqrt{d_{model}}$ 을 곱하지 않으면 학습이 어떻게 달라지는가?
4. 학습 가능한 위치 임베딩의 max_len을 1024로 설정하고 학습한 후, 추론 시 2048 길이 시퀀스를 넣으면 어떻게 되는가?
5. LayerNorm의 ϵ 값을 10^{-5} 에서 10^{-1} 로 바꾸면 출력이 어떻게 변하는가?

다음 장에서는 이 임베딩을 입력으로 받아 “각 토큰이 다른 토큰에 얼마나 주의를 기울일지”를 학습하는 어텐션 메커니즘을 구현한다.

Chapter 5

어텐션 메커니즘

이 장에서는 트랜스포머의 핵심인 **어텐션(attention)** 메커니즘을 구현한다. Scaled Dot-Product Attention에서 시작해 Multi-Head Attention까지 단계적으로 살펴본다.

5.1 직관: 어텐션이란 무엇인가

문장 “The cat sat on the mat because it was tired”에서 “it”이 무엇을 가리키는가? 사람은 쉽게 “cat”임을 안다. 컴퓨터는 어떻게 알 수 있을까? 어텐션은 이 문제를 푸는 메커니즘이다. 각 단어가 다른 모든 단어에 “얼마나 주의를 기울일지”를 학습하여, “it”이 “cat”에 높은 가중치를 주도록 만드는 것이다.

수식으로, 어텐션은 주어진 쿼리(query) Q 에 대해, 모든 키(key) K 와의 유사도를 계산하고, 그 유사도로 값(value) V 를 가중 합산하는 연산이다.

어텐션의 일반적 정의

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

- Q : 쿼리 (지금 보고 있는 토큰의 표현)
- K : 키 (다른 토큰들의 “라벨”)
- V : 값 (다른 토큰들의 “내용”)
- d_k : 키의 차원. $\sqrt{d_k}$ 로 나누는 이유는 dot product가 커져 softmax가 포화되는 것을 방지하기 위함.

5.1.1 직관적 비유: 도서관 검색

도서관에서 책을 찾는 상황을 상상하자. 당신이 원하는 정보(쿼리 Q)를 가지고 도서관에 간다. 각 책에는 제목과 키워드(키 K)가 있고, 내용(값 V)이 있다. 도서관 시스템은 당신의 쿼리를 각 책의 키와 비교하여 유사도를 계산하고, 가장 유사한 책의 내용을 가중 합산하여 답을 준다.

어텐션은 이 과정을 미분 가능하게 만든 것이다. “유사도”를 dot product로, “가중 합산”을 softmax 가중치로 수행한다.

5.2 Q, K, V 는 어디서 오는가

트랜스포머에서 Q, K, V 는 모두 같은 입력 x 에서 선형 변환으로 만들어진다.

$$Q = xW_Q, \quad K = xW_K, \quad V = xW_V$$

여기서 W_Q, W_K, W_V 는 학습 가능한 행렬. 이렇게 하면 같은 입력에 대해 “질문”, “라벨”, “내용” 세 가지 관점을 학습할 수 있다.

왜 세 개의 다른 행렬인가?

같은 x 를 세 번 사용하지 않고 W_Q, W_K, W_V 로 변환하는 이유는 “역할 분담” 때문이다. 쿼리는 “무엇을 찾을지”, 키는 “무엇이 있는지”, 값은 “실제 내용”을 표현해야 한다. 같은 벡터로 세 역할을 모두 하기 어렵다. 선형 변환을 통해 각 역할에 맞는 “표상 공간”으로 투영하는 것이다.

5.3 스케일링: 왜 $\sqrt{d_k}$ 로 나누는가?

$Q \cdot K^T$ 는 d_k 차원 벡터들의 내적이다. d_k 가 크면 내적 값이 커진다. Q, K 의 원소가 평균 0, 분산 1이라고 하면, 내적 $\sum_{i=1}^{d_k} Q_i K_i$ 의 분산은 d_k 다.

내적 값이 크면 softmax 출력이 원-핫에 가까워진다 (한 값이 1, 나머지 0). 이렇게 되면 그래디언트가 거의 0이 되어 학습이 멈춘다. $\sqrt{d_k}$ 로 나누면 분산이 1이 되어 softmax가 부드럽게 분포한다.

스케일링의 효과

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \quad \text{vs} \quad \text{softmax}(QK^T)$$

스케일링 없으면 $d_k = 64$ 일 때 내적 값이 ± 64 까지 갈 수 있어 softmax가 포화. 스케일링하면 ± 8 범위로 줄어 부드러운 분포.

5.4 Scaled Dot-Product Attention 구현

가장 기본 형태부터 시작하자. PyTorch 2.0의 `F.scaled_dot_product_attention`(SDPA)을 사용하면 메모리 효율적이고 빠르다.

```

1 import torch
2 import torch.nn.functional as F
3
4 class ScaledDotProductAttention(torch.nn.Module):
5     def __init__(self, dropout: float = 0.0):
6         super().__init__()
7         self.dropout_p = dropout
8
9     def forward(self, q, k, v, mask=None):
10        # q, k, v: [batch, n_heads, seq, d_head]
11        # mask: [batch, 1, seq, seq] 또는 [seq, seq]
12        out = F.scaled_dot_product_attention(
13            q, k, v, attn_mask=mask,
14            dropout_p=self.dropout_p if self.training else 0.0,
15        )
16        return out

```

SDPA는 C++ 레벨에서 최적화되어 있어, 파이썬 루프로 softmax를 직접 구현하는 것보다 수십 배 빠르다. 또한 Flash Attention 같은 최신 알고리즘을 자동으로 활용한다.

Flash Attention

Flash Attention (Dao et al. 2022)은 어텐션 계산을 GPU 메모리 계층 구조에 맞게 재설계하여, 메모리 접근을 줄이고 속도를 2~4배 향상시킨 알고리즘이다. PyTorch 2.0의 SDPA는 자동으로 Flash Attention을 활성화한다. 우리가 직접 구현할 필요 없이 `F.scaled_dot_product_attention` 한 줄로 혜택을 받는다.

5.5 인과 마스크 (Causal Mask)

자기회귀 언어 모델에서는 “미래”를 보면 안 된다. 토큰 t 를 예측할 때 토큰 $t+1, t+2, \dots$ 를 보면 안 된다는 뜻이다. 이를 위해 어텐션 점수 행렬의 상삼각(주대각선 위)을 $-\infty$ 로 마스킹하여 softmax 후 0이 되도록 한다.

```
1 def make_causal_mask(seq_len: int, device="cpu") -> torch.Tensor:
2     """하삼각 마스크. 미래 토큰을 -로 inf 마스킹."""
3     mask = torch.full((seq_len, seq_len), float("-inf"), device=device)
4     mask = torch.triu(mask, diagonal=1) # 주대각선 위를 -로 inf
5     return mask
```

		token 0		token 4	
token 0	0	-in	-in	-in	-inf
	0	0	-in	-in	-inf
	0	0	0	-in	-inf
	0	0	0	0	-inf
token 4	0	0	0	0	0

Figure 5.1: 인과 마스크 (5×5). 파란색 = 어텐션 허용(0), 빨간색 = 미래 토큰 차단($-\infty$). 행 i 는 토큰 i 가 열 j 토큰에 얼마나 주의를 기울일지 나타냄.

마스크를 어텐션 점수에 더하면, 마스킹된 위치의 점수는 $-\infty$ 가 되고, softmax 후에는 0이 된다. 즉 미래 토큰의 V 가 출력에 기여하지 않게 된다.

5.5.1 왜 $-\infty$ 인가?

softmax의 수식 $\text{softmax}(x_i) = e^{x_i} / \sum_j e^{x_j}$ 에서 $x_i = -\infty$ 면 $e^{-\infty} = 0$ 이 된다. 따라서 마스킹된 위치의 가중치가 정확히 0이 된다. 단순히 0을 더하는 것과 다르다: 0을 더하면 원래 점수가 그대로 남아 softmax 후 여전히 양수가 된다.

5.6 Multi-Head Attention

단일 어텐션은 하나의 “관점”만 학습할 수 있다. 하지만 언어에는 다양한 관계가 있다: 문법적 관계(주어-동사), 의미적 관계(동의어, 반의어), 담화 관계(지시어 참조 해결) 등. **멀티헤드 어텐션(Multi-Head Attention)**은 이를 여러 헤드로 분할하여 병렬 처리한다.

5.6.1 아이디어

d_{model} 차원을 h 개의 헤드로 분할하여, 각 헤드가 $d_{head} = d_{model}/h$ 차원에서 독립적으로 어텐션을 수행한다. 예를 들어 $d_{model} = 512, h = 8$ 이면 $d_{head} = 64$.

Multi-Head Attention

$$\text{MHA}(x) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$

각 헤드의 출력을 이어붙인 후 W_O 로 선형 변환.

5.6.2 왜 헤드를 나누는가?

$d_{\text{model}} = 512$ 인 단일 어텐션과 $d_{\text{head}} = 64, h = 8$ 인 멀티헤드 어텐션은 총 파라미터 수가 같다. 하지만 멀티헤드는 각 헤드가 다른 패턴을 학습할 수 있다. 예를 들어:

- 헤드 1: 주어-동사 관계
- 헤드 2: 형용사-명사 수식
- 헤드 3: 지시어 참조
- 헤드 4: 문장 경계

실제로 어텐션 헤드를 시각화해보면 (예: BertViz), 서로 다른 헤드가 확실히 다른 패턴에 주목하는 것을 볼 수 있다.

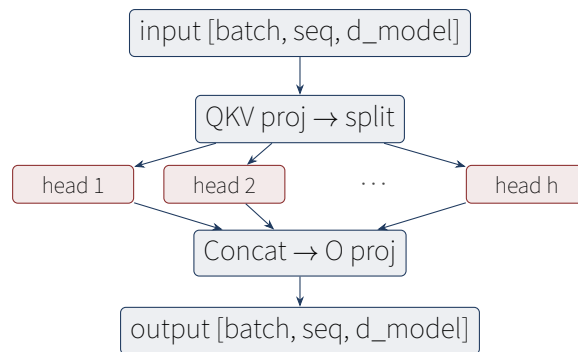


Figure 5.2: Multi-Head Attention 구조. 입력을 h 개 헤드로 분할하여 병렬 어텐션 수행 후 합침.

5.6.3 구현

효율성을 위해 Q, K, V 를 한 번에 계산한다. 입력 x 에 대해 $3d_{\text{model}}$ 차원으로 프로젝션한 뒤, 헤드별로 분할한다.

```

1 class MultiHeadAttention(nn.Module):
2     def __init__(self, config: ModelConfig):
3         super().__init__()
4         self.d_model = config.d_model
5         self.n_heads = config.n_heads
6         self.d_head = config.d_model // config.n_heads
7         # Q, K, 를 V 한 번에 계산 결합 (프로젝션)
8         self.qkv_proj = nn.Linear(config.d_model, 3 * config.d_model, bias=config.use_bias)
9         self.o_proj = nn.Linear(config.d_model, config.d_model, bias=config.use_bias)
10        self.attention = ScaledDotProductAttention(dropout=config.dropout)
11        self.dropout = nn.Dropout(config.dropout)
12
13    def forward(self, x, mask=None):
14        batch, seq, _ = x.shape
  
```

```

15     # [batch, seq, 3*d_model] -> [batch, seq, 3, n_heads, d_head]
16     qkv = self.qkv_proj(x).view(batch, seq, 3, self.n_heads, self.d_head)
17     # [3, batch, n_heads, seq, d_head]
18     qkv = qkv.permute(2, 0, 3, 1, 4)
19     q, k, v = qkv[0], qkv[1], qkv[2]
20
21     if mask is not None and mask.dim() == 2:
22         mask = mask.unsqueeze(0).unsqueeze(0) # [1, 1, seq, seq]
23
24     # 어텐션: [batch, n_heads, seq, d_head]
25     attn_out = self.attention(q, k, v, mask=mask)
26     # 헤드 합치기: [batch, seq, d_model]
27     attn_out = attn_out.transpose(1, 2).contiguous().view(batch, seq, self.d_model)
28     return self.dropout(self.o_proj(attn_out))

```

왜 QKV를 한 번에 계산하는가?

세 개의 `nn.Linear`를 따로 쓰는 대신 $3 * d_{model}$ 출력을 가진 하나의 `nn.Linear`를 쓰는 이유는 효율성이다. GPU는 큰 행렬 연산을 한 번에 수행하는 것에 최적화되어 있다. 세 번의 작은 연산보다 한 번의 큰 연산이 더 빠르다. 분할은 `view + permute`로 메모리 재구성 없이 수행된다.

5.7 인과성 테스트

어텐션 모듈의 가장 중요한 테스트는 “인과 마스크가 실제로 미래 토큰을 차단하는가”다. 다음 테스트는 이를 검증한다.

```

1 def test_attention_is_causal(small_config):
2     """위치 i의 출력이 위치 i+1 이후에 영향을 받지 않아야 함."""
3     attn = MultiHeadAttention(small_config)
4     attn.eval()
5     x = torch.randn(1, 6, small_config.d_model)
6     mask = make_causal_mask(6)
7     out1 = attn(x, mask=mask)
8     # i의 마지막 위치를 바꿔도 앞 위치의 출력은 변하지 않아야 함
9     x2 = x.clone()
10    x2[:, -1] = torch.randn(1, small_config.d_model)
11    out2 = attn(x2, mask=mask)
12    # 앞 5개 위치는 동일해야 함
13    assert torch.allclose(out1[:, :5], out2[:, :5], atol=1e-5)

```

이 테스트가 통과하면 인과 마스크가 올바르게 작동하고 있음을 확신할 수 있다. 만약 실패한다면 마스크가 뒤집혀 있거나 차원이 잘못된 것이다.

5.8 Self-Attention vs Cross-Attention

이 장에서 구현한 것은 자기 어텐션(self-attention)으로, Q, K, V 가 모두 같은 시퀀스에서 온다. 번역 모델의 디코더에서는 교차 어텐션(cross-attention)도 사용하는데, 이는 Q 는 디코더에서, K, V 는 인코더에서 온다. GPT는 디코더 only 구조라 자기 어텐션만 사용한다.

5.8.1 교차 어텐션은 어떻게 다른가?

교차 어텐션에서 $Q = x_{dec}W_Q$, $K = x_{enc}W_K$, $V = x_{enc}W_V$ 다. 디코더의 각 위치가 인코더 출력의 모든 위치를 참조한다. 번역에서 디코더가 “번역할 문장”을 생성할 때 원문의 해당 단어를 찾는 과정이다.

GPT는 이 교차 어텐션이 없다. 오직 자기 어텐션만으로 다음 토큰을 예측한다. 이것이 GPT를 “디코더 only” 모델이라 부르는 이유다.

5.9 어텐션 계산량 분석

어텐션의 계산량은 $O(L^2 \cdot d)$ 다. L 은 시퀀스 길이. 시퀀스가 길어지면 제곱으로 증가한다.

시퀀스 길이	어텐션 행렬 크기	FP16 메모리 (단일 헤드)
512	512×512	0.5 MB
2048	2048×2048	8 MB
8192	8192×8192	128 MB
32768	32768×32768	2 GB

이것이 긴 문맥 처리가 어려운 이유다. 2권 7장에서 KV cache와 PagedAttention으로 이 비용을 줄이는 방법을 다룬다.

5.10 실습: 어텐션 가중치 시각화

학습된 모델의 어텐션 가중치를 시각화하면 모델이 어떤 패턴에 주목하는지 볼 수 있다.

```
1 import matplotlib.pyplot as plt
2 import math
3 from makellm.model.attention import make_causal_mask
4
5 # 어텐션 가중치 수동 계산 및 시각화 (는SDPA 가중치를 내부적으로만 관리하므로 디버그 시 직접 계산)
6 model.eval()
7 with torch.no_grad():
8     x = torch.randn(1, 10, model.config.d_model) # [batch=1, seq=10, d_model]
9     x_norm = model.blocks[-1].ln1(x)
10    qkv = model.blocks[-1].attn.qkv_proj(x_norm)
11    batch, seq, _ = x.shape
12    qkv = qkv.view(batch, seq, 3, model.config.n_heads, model.config.d_head).permute(2, 0, 3, 1, 4)
13    q, k = qkv[0], qkv[1] # [1, n_heads, seq, d_head]
14
15    # 번째0 헤드의 가중치 계산: (Q @ K^T) / sqrt(d_head)
16    scores = (q[0, 0] @ k[0, 0].T) / math.sqrt(model.config.d_head)
17    mask = make_causal_mask(seq)
18    attn_weights = torch.softmax(scores + mask, dim=-1) # [seq, seq]
19
20 plt.imshow(attn_weights.numpy(), cmap='viridis')
21 plt.colorbar()
22 plt.xlabel('key position')
23 plt.ylabel('query position')
24 plt.title('Attention weights (head 0, last layer)')
```

5.11 요약

- 어텐션은 쿼리-키 유사도로 값을 가중 합산하는 연산이다.
- $\sqrt{d_k}$ 로 스케일링하여 softmax 포화를 방지한다.
- 인과 마스크로 미래 토큰을 차단하여 자기회귀 모델이 된다.
- 멀티헤드는 d_{model} 을 h 개 헤드로 분할하여 다양한 관계를 학습.
- QKV를 한 번에 계산하여 GPU 효율을 극대화.
- PyTorch 2.0의 SDPA를 활용하면 최적화된 어텐션을 한 줄로 사용 가능.
- 어텐션 계산량은 $O(L^2)$ 로 긴 시퀀스에서 병목.

5.12 연습 문제

1. $\sqrt{d_k}$ 스케일링을 빼면 학습이 어떻게 달라지는가? $d_k = 64$ 일 때 softmax 출력의 분포를 계산해보라.
2. 인과 마스크를 씌우지 않으면 어떤 일이 일어나는가? 학습 시와 추론 시 각각 설명하라.
3. 헤드 수 h 를 1로 하면 (단일 헤드) 멀티헤드와 어떤 차이가 있는가?
4. 헤드 수 h 를 d_{model} 과 같게 하면 (즉 $d_{head} = 1$) 어떤 일이 일어나는가?
5. 어텐션 행렬 $L \times L$ 의 메모리가 $L = 100,000$ 일 때 얼마인지 계산하라 (FP16 기준).

다음 장에서는 이 어텐션을 포함한 트랜스포머 블록을 조립한다. 어텐션 + FFN + 잔차 + Layer-Norm의 조합이 트랜스포머를 만든다.

Chapter 6

트랜스포머 블록 조립하기

이 장에서는 5장의 어텐션과 4장의 임베딩을 결합하여 **트랜스포머 블록(transformer block)**을 만든다. 어텐션만으로는 부족하고, FFN(Feed-Forward Network), 잔차 연결(residual connection), 레이어 정규화(layer normalization)가 함께 있어야 비로소 의미 있는 표현을 학습할 수 있다.

6.1 트랜스포머 블록의 구조

하나의 트랜스포머 블록은 다음 네 가지 컴포넌트로 이루어진다:

1. **Multi-Head Attention**: 토큰 간 정보 교환
2. **Feed-Forward Network (FFN)**: 위치별 비선형 변환
3. **Residual Connection**: 입력을 출력에 더하여 깊은 네트워크 학습
4. **Layer Normalization**: 학습 안정화

이 네 가지가 어떻게 조합되는지가 트랜스포머 블록의 핵심이다. 1권에서는 Pre-LN 구조(GPT-2 스타일)를 사용한다.

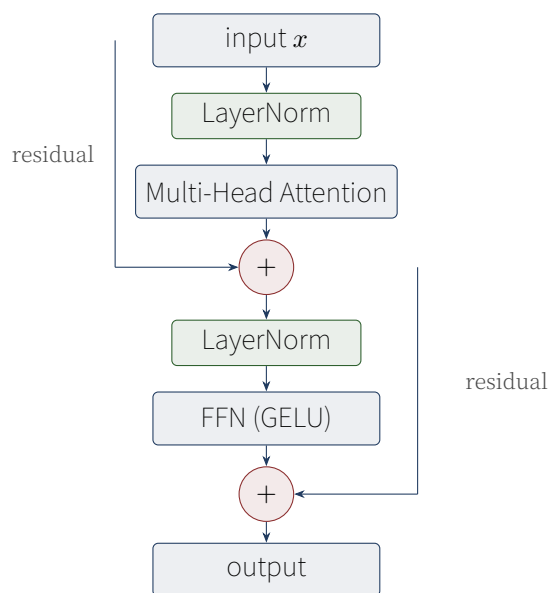


Figure 6.1: Pre-LN 트랜스포머 블록 구조. 어텐션과 FFN 각각에 잔차 연결과 LayerNorm이 있음.

6.2 Feed-Forward Network (FFN)

어텐션은 토큰 간 “수평적” 정보 교환을 수행한다. 반면 FFN은 각 토큰 위치에서 “수직적” 비선형 변환을 수행한다. 구조는 단순하다:

$$\text{FFN}(x) = \text{Linear}_2(\text{GELU}(\text{Linear}_1(x)))$$

Linear_1 은 $d_{\text{model}} \rightarrow d_{\text{ff}}$ 로 확장하고, Linear_2 는 $d_{\text{ff}} \rightarrow d_{\text{model}}$ 로 다시 축소한다. d_{ff} 는 보통 $4d_{\text{model}}$ 이며, 이 “확장-축소” 패턴이 비선형 표현력을 준다.

6.2.1 직관: 왜 확장-축소인가?

$d_{\text{model}} = 512$ 를 $d_{\text{ff}} = 2048$ 로 확장했다가 다시 512로 축소하는 이유는 “더 큰 표현 공간에서 연산”하기 위해서다. 고차원 공간에서는 더 복잡한 결정 경계를 그릴 수 있다. 활성화 함수(GELU)의 비선형성과 결합하여, FFN은 단순한 선형 변환 이상의 것을 학습한다.

실제로 FFN은 “키-값 메모리” 역할을 한다는 연구 결과(Geva et al. 2021)도 있다. Linear_1 의 행벡터가 “키”, Linear_2 의 열벡터가 “값”으로 동작하여, FFN이 사실 정보 검색 메커니즘이라는 해석이다.

```
1 import torch.nn.functional as F
2
3 class FeedForward(nn.Module):
4     def __init__(self, d_model: int, d_ff: int, dropout: float = 0.1):
5         super().__init__()
6         self.w1 = nn.Linear(d_model, d_ff)
7         self.w2 = nn.Linear(d_ff, d_model)
8         self.dropout = nn.Dropout(dropout)
9         self._init_weights()
10
11     def _init_weights(self) -> None:
12         nn.init.normal_(self.w1.weight, mean=0.0, std=0.02)
13         nn.init.normal_(self.w2.weight, mean=0.0, std=0.02)
14         if self.w1.bias is not None:
15             nn.init.zeros_(self.w1.bias)
16         if self.w2.bias is not None:
17             nn.init.zeros_(self.w2.bias)
18
19     def forward(self, x: torch.Tensor) -> torch.Tensor:
20         """x: [batch, seq, d_model] -> [batch, seq, d_model]"""
21         return self.dropout(self.w2(F.gelu(self.w1(x))))
```

6.3 GELU vs ReLU

원본 트랜스포머는 ReLU를 사용했지만, GPT-2 이후로는 **GELU(Gaussian Error Linear Unit)**가 표준이다. GELU는 ReLU의 “냉혹한” 0/1 임계점 대신 가우시안 누적 분포를 사용하여 부드럽게 전환한다.

GELU

$$\text{GELU}(x) = x \cdot \Phi(x) \approx 0.5x \left(1 + \tanh \left[\sqrt{2/\pi} (x + 0.044715x^3) \right] \right)$$

$\Phi(x)$ 는 표준정규분포의 누적분포함수. $x < 0$ 에서도 작은 음수값을 허용하여 그래디언트 흐름이 더 좋다.

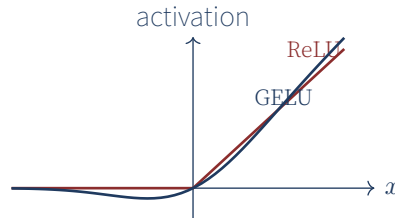


Figure 6.2: ReLU vs GELU. ReLU는 $x < 0$ 에서 정확히 0이지만, GELU는 작은 음수값을 허용하여 부드럽게 전환.

6.3.1 왜 GELU가 더 나은가?

ReLU는 $x < 0$ 에서 완전히 0이 되어 그래디언트도 0이 된다 (“죽은 ReLU” 문제). GELU는 작은 음수값을 허용하여 그래디언트가 항상 흐른다. 또한 전환점이 부드러워 미분 가능하여 최적화가 안정적이다.

실제로 GPT-2, BERT, RoBERTa 등 대부분의 현대 모델이 GELU를 사용한다. 2권에서는 SwiGLU라는 더 발전된 활성화 함수를 다룬다.

6.4 잔차 연결 (Residual Connection)

잔차 연결(residual connection)은 입력을 출력에 직접 더하는 연산이다: $y = x + f(x)$. 이 단순한 아이디어가 깊은 네트워크 학습을 가능하게 했다. He et al. (2016)의 ResNet이 이미지 분류에서 100층 이상 학습을 가능하게 한 것이 그 시작이다.

6.4.1 직관: 왜 잔차가 도움인가?

잔차가 없으면 네트워크는 $f(x) = y$ 를 학습해야 한다. 깊어질수록 이 매핑을 학습하기 어렵다. 잔차가 있으면 $f(x) = y - x$, 즉 “잔차”만 학습하면 된다. 입력 x 가 이미 y 와 가깝다면 f 는 작은 보정만 하면 된다.

또한 역전파 시 그래디언트가 “지름길”을 통해 직접 흐른다. $y = x + f(x)$ 의 미분은 $\frac{dy}{dx} = 1 + f'(x)$ 다. $f'(x) = 0$ 이어도 그래디언트는 1이 되어 사라지지 않는다.

트랜스포머 블록에는 두 개의 잔차 연결이 있다: 어텐션 주변 하나, FFN 주변 하나. GPT는 12~96층이므로 잔차 없이는 학습이 불가능하다.

잔차 없는 100층 네트워크

실험적으로 잔차를 빼고 100층 네트워크를 학습하면 손실이 줄어들지 않는다. 그래디언트가 역전파 도중 사라져 앞쪽 층이 학습되지 않기 때문이다. 잔차 연결은 단순하지만 깊은 네트워크의 필수 요소다.

6.5 Pre-LN vs Post-LN

LayerNorm의 위치에 따라 두 가지 변형이 있다.

6.5.1 Post-LN (Vaswani 원본)

원본 트랜스포머는 LayerNorm을 어텐션/FFN 뒤에 배치했다:

$$x = \text{LN}(x + \text{Attn}(x))$$

하지만 이 구조는 깊은 네트워크에서 학습이 불안정하다. 초기화에 매우 민감하고 warmup이 없으면 발산하기 쉽다.

6.5.2 Pre-LN (GPT-2 스타일)

LayerNorm을 어텐션/FFN 앞에 배치한다:

$$x = x + \text{Attn}(\text{LN}(x))$$

이 구조는 잔차 경로에 LayerNorm이 없어 그래디언트가 더 잘 흐른다. 깊은 네트워크에서도 warmup 없이 안정적으로 학습되어 현대 LLM의 표준이다.

6.5.3 왜 Pre-LN이 더 안정적인가?

Post-LN에서 잔차 경로 $x \rightarrow x + \text{Attn}(x) \rightarrow \text{LN}(\cdot)$ 를 거치면, 그래디언트가 LayerNorm을 통과해야 한다. LayerNorm은 분산으로 나누는 연산이라 그래디언트를 스케일링하는데, 깊은 네트워크에서 이 스케일링이 누적되면 그래디언트가 사라질 수 있다.

Pre-LN은 잔차 경로 $x \rightarrow x + \text{Attn}(\text{LN}(x))$ 에서 x 가 LayerNorm을 거치지 않고 직접 흐른다. 어텐션 출력만 LayerNorm을 거친 후 더해진다. 그래디언트가 identity 경로를 통해 그대로 흘러 깊은 네트워크에서도 학습이 가능하다.

```
1 class TransformerBlock(nn.Module):
2     def __init__(self, config: ModelConfig, layer_norm_style: str = "pre"):
3         super().__init__()
4         self.layer_norm_style = layer_norm_style
5         self.ln1 = nn.LayerNorm(config.d_model, eps=config.layer_norm_eps)
6         self.ln2 = nn.LayerNorm(config.d_model, eps=config.layer_norm_eps)
7         self.attn = MultiHeadAttention(config)
8         self.ffn = FeedForward(config.d_model, config.d_ff, config.dropout)
9
10    def forward(self, x, mask=None):
11        if self.layer_norm_style == "pre":
12            # Pre-LN: 정규화 -> 어텐션 -> 잔차
13            x = x + self.attn(self.ln1(x), mask=mask)
14            x = x + self.ffn(self.ln2(x))
15        else:
16            # Post-LN: 어텐션 -> 잔차 -> 정규화
17            x = self.ln1(x + self.attn(x, mask=mask))
18            x = self.ln2(x + self.ffn(x))
19        return x
```


Pre-LN을 써야 하는 이유

Post-LN은 초기 그래디언트가 잔차 경로가 아닌 LayerNorm을 통해서만 흐르기 때문에, 깊은 네트워크에서 사라질 수 있다. Pre-LN은 잔차 경로에 아무것도 없어 그래디언트가 identity하게 흘러 100층 이상에서도 학습이 가능하다. 단, Pre-LN은 마지막에 추가 LayerNorm이 필요하다 (다음 장에서 In_f로 구현).

6.6 잔차 스케일링

GPT-2는 잔차 스트림에 직접 기여하는 가중치(어텐션의 W_O , FFN의 W_2)를 더 작게 초기화한다. 층이 깊어질수록 잔차 출력의 분산이 커지는 것을 방지하기 위함이다.

```
1 def _init_residuals(self, module: nn.Module) -> None:
2     """잔차 스트림에 직접 기여하는 프로젝션 가중치를 작게 초기화."""
3     if isinstance(module, nn.Linear):
4         if module.weight.shape[0] == self.config.d_model:
5             # 잔차로 들어가는 출력: 더 작은 표준편차로 초기화
6             nn.init.normal_(module.weight, mean=0.0,
7                             std=0.02 / math.sqrt(2 * self.config.n_layers))
```

$1/\sqrt{2N}$ 스케일링은 N 층일 때 잔차 출력의 분산이 층 수에 비례해 커지는 것을 보정한다. N 이 클수록 더 작은 초기화가 필요하다.

6.6.1 왜 $2N$ 인가?

트랜스포머 블록마다 잔차에 기여하는 출력이 2개 (어텐션, FFN)다. N 개 블록이면 총 $2N$ 개의 출력이 잔차에 더해진다. 각 출력의 분산이 σ^2 이면 합인 분산은 $2N\sigma^2$ 이다. 이것이 1이 되려면 $\sigma = 1/\sqrt{2N}$ 이 되어야 한다.

6.7 블록 테스트

```
1 def test_block_shape(small_config):
2     block = TransformerBlock(small_config)
3     x = torch.randn(2, 8, small_config.d_model)
4     out = block(x)
5     assert out.shape == (2, 8, small_config.d_model)
6
7 def test_block_residual_connection(small_config):
8     """잔차 연결이 있으면 입력과 출력의 차이가 유한해야 함."""
9     block = TransformerBlock(small_config)
10    block.eval()
11    x = torch.randn(2, 8, small_config.d_model)
12    out = block(x)
13    diff = (out - x).abs().mean()
14    assert diff.item() < 5.0 # 잔차가 너무 크면 정규화 문제
15
16 def test_feedforward_shape(small_config):
17     ffn = FeedForward(small_config.d_model, small_config.d_ff)
18     x = torch.randn(2, 8, small_config.d_model)
19     out = ffn(x)
20     assert out.shape == x.shape
```

6.8 트랜스포머 블록의 계산량

블록 하나의 FLOPs는 대략:

- QKV 프로젝트: $3 \cdot L \cdot d^2$
- 어텐션 점수: $L^2 \cdot d$
- 어텐션 출력 프로젝트: $L \cdot d^2$
- FFN: $2 \cdot L \cdot d \cdot d_{ff} = 8Ld^2$ (since $d_{ff} = 4d$)

총 $O(L \cdot d^2)$ 이며, 어텐션의 $O(L^2d)$ 는 $L < d$ 일 때 무시할 수 있다. 실제 GPT-3 ($d = 12288$, $L = 2048$)에서 $L \cdot d^2 = 3 \times 10^{11}$, $L^2 \cdot d = 5 \times 10^{10}$ 이라 FFN이 더 지배적이다.

6.9 요약

- 트랜스포머 블록 = 어텐션 + FFN + 잔차 연결 + LayerNorm.
- FFN은 위치별 비선형 변환 (Linear $\rightarrow$ GELU $\rightarrow$ Linear).
- GELU는 ReLU보다 부드러운 전환으로 학습 안정성 향상.
- 잔차 연결은 깊은 네트워크 학습의 핵심.
- Pre-LN이 Post-LN보다 학습 안정성이 좋아 현대 LLM 표준.
- 잔차 스케일링 $1/\sqrt{2N}$ 로 깊은 네트워크의 분산 폭발을 방지.

6.10 연습 문제

1. FFN의 d_{ff} 를 $4d_{model}$ 대신 d_{model} 로 하면 어떤 일이 일어나는가?
2. 잔차 연결을 제거하고 12층 모델을 학습하면 어떤 현상이 관찰되는가?
3. Pre-LN에서 마지막 블록 후에 추가 LayerNorm이 필요한 이유는?
4. GELU 대신 Sigmoid Linear Unit (SiLU, $x \cdot \sigma(x)$)를 사용하면 어떤 차이가 있는가?
5. 잔차 스케일링 없이 100층 모델을 초기화하면 학습 초기에 어떤 일이 일어나는가?

다음 장에서는 이 블록을 N 층 쌓아 올려 완전한 GPT 모델을 만들고, 학습 루프를 구축한다.

Chapter 7

GPT 모델 완성과 학습 루프

이 장에서는 6장의 트랜스포머 블록을 N 층 쌓아 올려 완전한 **GPT 모델**을 만들고, 이 모델을 학습하는 루프를 구축한다. 데이터셋, 손실 함수, 옵티마이저, 스케줄러를 모두 직접 구현한다.

7.1 GPT 모델 구조

GPT는 트랜스포머 디코더를 N 층 쌓은 자기회귀 언어 모델이다. 전체 구조는 다음과 같다:

1. 토큰 ID $\rightarrow$ [Token + Position Embedding]

Transformer Block $\times N$

Final LayerNorm

Head: $d_{model} \rightarrow V$

2. logits

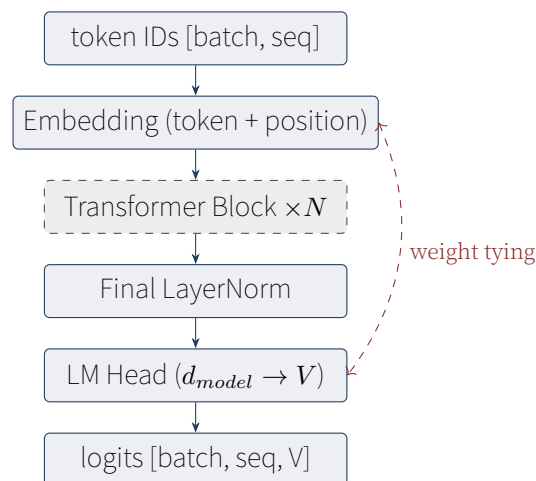


Figure 7.1: GPT 모델 구조. 임베딩 가중치와 LM Head 가중치를 공유(weight tying)하여 파라미터 절약.

7.2 가중치 타이 (Weight Tying)

임베딩 행렬 E ($V \times d_{model}$)과 LM Head 행렬 W_{out} ($d_{model} \times V$)은 사실 같은 정보를 담고 있다. 토큰 i 의 임베딩은 $E[i]$ 이고, 로짓 계산 시 $W_{out}^T[i]$ 가 사용된다. 이 둘을 공유하면 파라미터 수를 $V \times d_{model}$ 만큼 절약할 수 있다.

```
1 class GPT(nn.Module):
2     def __init__(self, config: ModelConfig):
3         super().__init__()
```

```

4     self.config = config
5     self.embedding = EmbeddingStack(
6         vocab_size=config.vocab_size,
7         d_model=config.d_model,
8         max_len=config.context_length,
9         pad_id=config.pad_token_id,
10    )
11    self.blocks = nn.ModuleList([
12        TransformerBlock(config, layer_norm_style="pre")
13        for _ in range(config.n_layers)
14    ])
15    self.ln_f = nn.LayerNorm(config.d_model, eps=config.layer_norm_eps)
16    self.lm_head = nn.Linear(config.d_model, config.vocab_size, bias=False)
17    # 가중치 타이
18    self.lm_head.weight = self.embedding.token_emb.embedding.weight

```

가중치 타이의 장단점

장점: 파라미터 절약 (GPT-2 small 기준 약 1/3 감소), 임베딩과 출력의 일관성.

단점: 임베딩이 출력 레이어의 그래디언트도 받아 학습이 더 복잡해짐. 일부 모델(LLaMA 등)은 타이를 사용하지 않고 별도 학습.

GPT-2, GPT-3는 타이 사용. LLaMA, Mistral은 미사용. 이 책에서는 GPT 스타일을 따라 타이를 사용한다.

7.3 Forward Pass

```

1     def forward(self, token_ids, return_hidden=False):
2         batch, seq = token_ids.shape
3         # 인과 마스크 생성
4         mask = make_causal_mask(seq, device=token_ids.device)
5         # 임베딩
6         x = self.embedding(token_ids)
7         # 트랜스포머 블록 통과
8         for block in self.blocks:
9             x = block(x, mask=mask)
10        # 최종 정규화 (Pre-이므로 LN 마지막에 필요)
11        x = self.ln_f(x)
12        # 로짓
13        logits = self.lm_head(x)
14        if return_hidden:
15            return logits, x
16        return logits

```

왜 마지막에 LayerNorm이 필요한가?

Pre-LN 블록은 각 블록의 입력을 정규화한다. 하지만 마지막 블록의 출력은 정규화되지 않은 상태로 잔차 스트림에 더해진다. 이것을 LM Head에 넣기 전에 정규화해야 출력 분포가 안정적이다. 그래서 `ln_f` (final LayerNorm)가 필요하다.

Post-LN에서는 각 블록 출력이 이미 정규화되어 있으므로 `ln_f`가 불필요하다.

7.4 데이터셋: 슬라이딩 윈도우

언어 모델링 데이터셋은 텍스트를 토큰화한 뒤, 고정 길이 청크로 분할한다. 각 청크에서 input은 처음 $L - 1$ 토큰, target은 마지막 $L - 1$ 토큰 (한 칸 shift).

```
1 class LMDataset(Dataset):
2     def __init__(self, text, tokenizer, context_length=128,
3                 add_bos=True, add_eos=True):
4         self.context_length = context_length
5         # 텍스트를 토큰화하여 긴 시퀀스로
6         if isinstance(text, str):
7             text = [text]
8         all_ids = []
9         for t in text:
10            ids = tokenizer.encode(t, add_bos=add_bos, add_eos=add_eos)
11            all_ids.extend(ids)
12        self._ids = all_ids
13        # context_length+1 크기 청크로 분할
14        cl = context_length + 1
15        self._chunks = []
16        for i in range(0, len(all_ids), cl):
17            chunk = all_ids[i:i+cl]
18            if len(chunk) < cl:
19                chunk = chunk + [tokenizer.special.pad_id] * (cl - len(chunk))
20            self._chunks.append(chunk)
21
22    def __getitem__(self, idx):
23        chunk = self._chunks[idx]
24        input_ids = torch.tensor(chunk[:-1], dtype=torch.long)
25        target_ids = torch.tensor(chunk[1:], dtype=torch.long)
26        return input_ids, target_ids
```

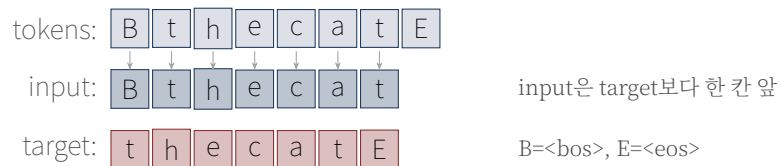


Figure 7.2: 슬라이딩 윈도우 데이터셋. input과 target은 같은 시퀀스를 한 칸 shift한 것. 모델은 input을 보고 target을 예측.

7.5 손실 함수: Cross-Entropy

언어 모델의 손실은 **교차 엔트로피(cross-entropy)**다. 모델이 예측한 확률 분포와 정답(원-핫) 사이의 차이를 측정한다.

Cross-Entropy Loss

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log P(y_i | x_{<i})$$

y_i 는 정답 토큰, P 는 모델의 예측 확률. 낮을수록 좋다.

```

1 def cross_entropy_loss(logits, targets, ignore_index=-100):
2     # logits: [batch, seq, V], targets: [batch, seq]
3     batch, seq, vocab = logits.shape
4     logits_flat = logits.view(-1, vocab)
5     targets_flat = targets.view(-1)
6     return F.cross_entropy(logits_flat, targets_flat, ignore_index=ignore_index)

```

7.5.1 Label Smoothing (선택)

정답에 100% 확신하는 대신 약간의 불확실성을 허용하여 일반화 성능을 높이는 기법.

```

1 def label_smoothing_loss(logits, targets, smoothing=0.1, ignore_index=-100):
2     """정답 토큰에 1-smoothing 확률, 나머지에 smoothing/vocab 확률."""
3     log_probs = F.log_softmax(logits.view(-1, logits.size(-1)), dim=-1)
4     targets_flat = targets.view(-1)
5     nll_loss = -log_probs.gather(1, targets_flat.clamp(min=0).unsqueeze(1)).squeeze(1)
6     smooth_loss = -log_probs.mean(dim=-1)
7     mask = (targets_flat != ignore_index).float()
8     nll_loss = (nll_loss * mask).sum() / mask.sum().clamp(min=1)
9     smooth_loss = (smooth_loss * mask).sum() / mask.sum().clamp(min=1)
10    return (1.0 - smoothing) * nll_loss + smoothing * smooth_loss

```

7.6 옵티마이저: AdamW

AdamW는 Adam에 weight decay를 결합한 최적화 알고리즘이다. 핵심은 바이어스와 LayerNorm에는 weight decay를 적용하지 않는 것.

```

1 def build_optimizer(model, config):
2     # 파라미터 그룹 분리: decay / no_decay
3     decay, no_decay = [], []
4     for name, p in model.named_parameters():
5         if not p.requires_grad: continue
6         if p.ndim < 2 or "bias" in name or "ln" in name.lower():
7             no_decay.append(p)
8         else:
9             decay.append(p)
10    return AdamW(
11        [{"params": decay, "weight_decay": config.weight_decay},
12         {"params": no_decay, "weight_decay": 0.0}],
13        lr=config.learning_rate, betas=(0.9, 0.95)
14    )

```

왜 바이어스와 LayerNorm은 weight decay에서 빼는가?

Weight decay는 가중치를 0으로 끌어당기는 정규화다. 행렬 가중치에는 과적합 방지 효과가 있지만, 바이어스와 LayerNorm의 γ, β 는 차원이 낮아 weight decay의 효과가 미미하고 오히려 학습을 방해할 수 있다. PyTorch 커뮤니티의 표준 관행이다.

7.7 학습률 스케줄러

학습 초기에는 학습률을 서서히 올리고(warmup), 이후 코사인 곡선으로 감소시킨다. 이 **warmup + cosine** 스케줄은 GPT-3에서 표준으로 사용된다.

```
1 def build_scheduler(optimizer, config, total_steps):
2     warmup = config.warmup_steps
3     def lr_lambda(step):
4         if step < warmup:
5             return float(step) / max(1, warmup)
6         progress = (step - warmup) / max(1, total_steps - warmup)
7         if config.lr_scheduler == "cosine":
8             return 0.5 * (1.0 + math.cos(math.pi * min(progress, 1.0)))
9         return 1.0
10    return LambdaLR(optimizer, lr_lambda)
```

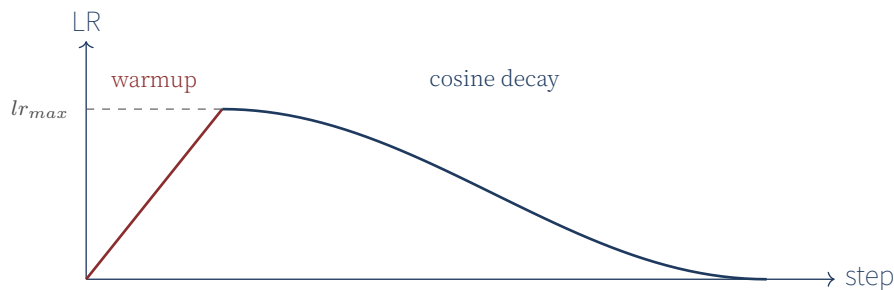


Figure 7.3: Warmup + Cosine 학습률 스케줄. 처음에는 선형으로 올리고, 이후 코사인 곡선으로 감소.

7.7.1 왜 warmup이 필요한가?

학습 초기에는 가중치가 무작위에 가깝다. 큰 학습률을 바로 적용하면 그래디언트가 불안정하여 발산할 수 있다. 작은 학습률로 시작하여 점진적으로 올리면 모델이 안정적인 영역으로 들어간다.

특히 어텐션의 초기 그래디언트는 매우 클 수 있는데 (softmax 포화 등), warmup이 이를 완화한다. Pre-LN 구조는 Post-LN보다 warmup에 덜 민감하지만, 여전히 사용하는 것이 안전하다.

7.8 그래디언트 클리핑

그래디언트가 너무 커서 가중치가 크게 변하는 것을 방지하기 위해 그래디언트 norm을 제한한다.

```
1 if config.grad_clip > 0:
2     torch.nn.utils.clip_grad_norm_(model.parameters(), config.grad_clip)
```

보통 `grad_clip = 1.0`을 사용한다. 그래디언트 norm이 1.0을 초과하면 1.0으로 스케일링한다.

7.9 트레이너

모든 컴포넌트를 묶어 학습 루프를 만든다.

```
1 class Trainer:
2     def __init__(self, model, train_dataset, config, model_config, eval_dataset=None):
3         self.model = model
4         self.config = config
```

```

5 self.device = torch.device(config.device)
6 self.model.to(self.device)
7 self.train_loader = DataLoader(train_dataset, batch_size=config.batch_size,
8                                 shuffle=True, collate_fn=collate_batch)
9
10 # 총 스텝 수 계산
11 total_steps = len(train_dataset) // config.batch_size * config.max_epochs
12 self.optimizer = build_optimizer(model, config)
13 self.scheduler = build_scheduler(self.optimizer, config, total_steps)
14 # ...
15
16 def train(self):
17     self.model.train()
18     for inputs, targets in self.train_loader:
19         inputs, targets = inputs.to(self.device), targets.to(self.device)
20         logits = self.model(inputs)
21         loss = cross_entropy_loss(logits, targets,
22                                   ignore_index=self.model_config.pad_token_id)
23         self.optimizer.zero_grad(set_to_none=True)
24         loss.backward()
25         if self.config.grad_clip > 0:
26             torch.nn.utils.clip_grad_norm_(self.model.parameters(),
27                                             self.config.grad_clip)
28         self.optimizer.step()
29         self.scheduler.step()

```

7.10 Perplexity

퍼플렉서티(Perplexity, PPL)는 언어 모델의 대표적 평가 지표다.

Perplexity

$$\text{PPL} = \exp \left(\frac{1}{N} \sum_i \log P(y_i | x_{<i}) \right) = \exp(\text{loss})$$

PPL은 모델이 다음 토큰을 예측할 때 “평균적으로 몇 개의 후보로 헤매는가”를 나타낸다. PPL=1이면 완벽, PPL=10이면 10개 후보로 좁힐 수 있다는 뜻.

PPL 해석

- PPL = 1: 모델이 다음 토큰을 100% 확신. 이론적 최소.
- PPL = 10: 10개 토큰 중 하나로 좁힘. 언어 모델로서 쓸만한 수준.
- PPL = 100: 100개 토큰 중 하나. 거의 무작위.
- PPL = V (어휘 크기): 완전 무작위. 학습 안 됨.

GPT-2 small의 WikiText-103 PPL은 약 17. GPT-3 175B는 약 11.

7.11 체크포인트 저장/로드

학습 중 모델 가중치를 주기적으로 저장하여, 중단 후 재개하거나 추론에 사용할 수 있다.

```

1 def save_checkpoint(self, filename):
2     path = self.out_dir / filename

```



```

3 torch.save({
4     "model_state": self.model.state_dict(),
5     "optimizer_state": self.optimizer.state_dict(),
6     "scheduler_state": self.scheduler.state_dict(),
7     "global_step": self.global_step,
8     "model_config": self.model_config.to_dict(),
9     "trainer_config": self.config.to_dict(),
10 }, path)
11
12 def load_checkpoint(self, path):
13     ckpt = torch.load(path, map_location=self.device)
14     self.model.load_state_dict(ckpt["model_state"])
15     self.optimizer.load_state_dict(ckpt["optimizer_state"])
16     self.scheduler.load_state_dict(ckpt["scheduler_state"])
17     self.global_step = ckpt["global_step"]

```

7.12 요약

- GPT = Embedding + Transformer Block $\times N$ + LayerNorm + LM Head.
- 가중치 타이로 임베딩과 LM Head 가중치를 공유하여 파라미터 절약.
- Pre-LN 구조에서는 마지막에 $\ln_f$ 가 필요.
- 언어 모델링은 슬라이딩 윈도우로 (input, target) 쌍 생성.
- 손실은 cross-entropy, 옵티마이저는 AdamW (bias/LayerNorm은 weight decay 제외).
- Warmup + Cosine 학습률 스케줄 + 그래디언트 클리핑이 표준.
- Perplexity = $\exp(\text{loss})$ 로 모델 품질 평가.

7.13 연습 문제

1. 가중치 타이를 사용하지 않으면 GPT-2 small ($V=50257$, $d=768$)에서 파라미터가 얼마나 늘어나는가?
2. Warmup 스텝 수를 0으로 하면 학습이 어떻게 달라지는가?
3. Label smoothing을 0.3으로 강하게 적용하면 어떤 효과가 있는가?
4. 그래디언트 클리핑 없이 학습하면 어떤 현상이 발생할 수 있는가?
5. PPL이 1.0인 모델이 실제로 존재할 수 있는가? 왜 그런가?

다음 장에서는 학습된 모델로 텍스트를 생성하는 방법을 다룬다. Greedy, Top-K, Top-P 샘플링 전부 구현한다.

Chapter 8

텍스트 생성 — 샘플링 전략과 디코딩

이 장에서는 학습된 GPT 모델로 텍스트를 생성하는 과정을 구현한다. 단순한 greedy 디코딩부터 Top-K, Top-P, Temperature 샘플링까지 다양한 전략을 다룬다.

8.1 자기회귀 생성의 원리

언어 모델의 생성은 **자기회귀(autoregressive)** 방식으로 이루어진다. 즉, 한 번에 토큰 하나씩 생성하고, 그 토큰을 다시 입력에 추가하여 다음 토큰을 생성하는 과정을 반복한다.

1. 프롬프트 $[w_1, \dots, w_k]$ 로 시작.
2. 모델에 넣어 다음 토큰 분포 $P(w_{k+1}|w_1, \dots, w_k)$ 를 얻음.
3. 분포에서 w_{k+1} 을 샘플링 (또는 argmax).
4. 시퀀스에 추가: $[w_1, \dots, w_k, w_{k+1}]$.
5. 종료 조건(<eos> 또는 최대 길이)까지 2~4 반복.

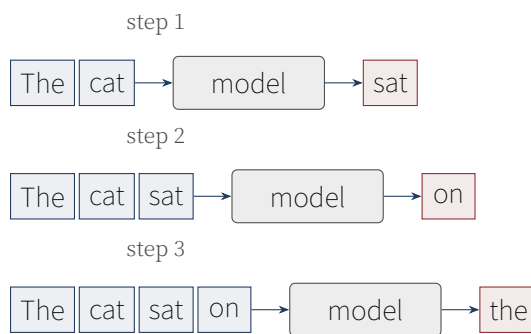


Figure 8.1: 자기회귀 생성 과정. 매 스텝마다 이전 토큰들을 모델에 넣고 다음 토큰을 예측.

8.2 Greedy 샘플링

가장 단순한 전략은 매 스텝 가장 확률이 높은 토큰을 선택하는 것이다.

```
1 class GreedySampler:
2     def __call__(self, logits: torch.Tensor) -> torch.Tensor:
3         # logits: [batch, vocab_size] -> token_ids: [batch]
4         return torch.argmax(logits, dim=-1)
```

Greedy는 결정론적이고 빠르지만, 매번 같은 토큰만 선택하여 단조로운 텍스트가 나오는 경향이 있다. “The cat sat on the mat. The cat sat on the mat. The cat...” 같은 반복에 빠지기 쉽다.

Greedy의 함정: 반복 루프

Greedy 디코딩은 “안전한” 토큰만 선택하여 재미없는 텍스트를 만든다. 더 심한 문제는 반복 루프에 빠진다는 것이다. “I went to the store to buy some store to buy some store to buy...” 같은 현상.

이유: 모델이 “store” 다음에 “to buy”를 높은 확률로 예측하고, “to buy” 다음에 “some store”를 높은 확률로 예측하는 루프가 형성되면, greedy는 거기서 빠져나오지 못한다.

8.3 Temperature 샘플링

분포의 “날카로움”을 조절하는 파라미터를 도입한다. 로짓을 온도 T 로 나눈 후 softmax.

Temperature Sampling

$$P_T(w) = \frac{\exp(\text{logit}(w)/T)}{\sum_{w'} \exp(\text{logit}(w')/T)}$$

- $T < 1$: 분포가 날카로워짐 (greedy에 가까워짐)
- $T > 1$: 분포가 평평해짐 (더 무작위)
- $T = 1$: 원래 분포

```
1 class TemperatureSampler:
2     def __init__(self, temperature: float = 1.0):
3         assert temperature > 0
4         self.temperature = temperature
5
6     def __call__(self, logits):
7         probs = F.softmax(logits / self.temperature, dim=-1)
8         return torch.multinomial(probs, num_samples=1).squeeze(-1)
```

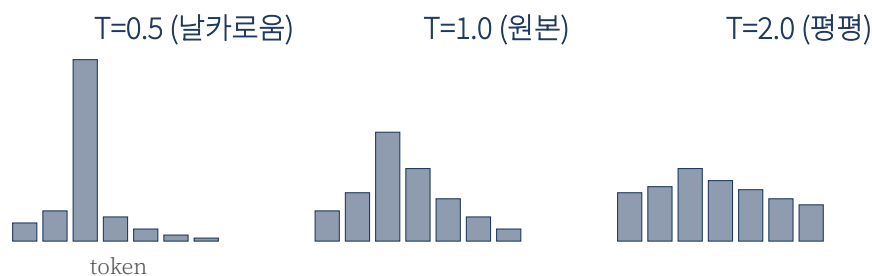


Figure 8.2: Temperature에 따른 분포 변화. T 가 낮으면 분포가 날카로워져 greedy에 가까워지고, T 가 높으면 평평해져 무작위성이 증가.

8.4 Top-K 샘플링

확률이 낮은 토큰이 선택되는 것을 방지하기 위해, 상위 K 개 토큰만 고려한다.

```
1 class TopKSampler:
2     def __init__(self, k: int = 50, temperature: float = 1.0):
3         self.k = k
4         self.temperature = temperature
5
```

```

6  def __call__(self, logits):
7      scaled = logits / self.temperature
8      top_values, top_indices = torch.topk(scaled, self.k, dim=-1)
9      probs = F.softmax(top_values, dim=-1)
10     sampled = torch.multinomial(probs, num_samples=1).squeeze(-1)
11     return top_indices.gather(-1, sampled.unsqueeze(-1)).squeeze(-1)

```

8.4.1 Top-K의 직관

로짓 분포에서 “꼬리”(확률이 매우 낮은 토큰)를 잘라낸다. 예를 들어 어휘가 5만 개일 때, 상위 50개만 고려하면 99.99%의 확률 질량을 가진 토큰들 중에서 샘플링하는 셈이다.

Top-K = 1이면 greedy와 동일. Top-K = V (어휘 크기)면 temperature 샘플링과 동일.

8.5 Top-P (Nucleus) 샘플링

Top-K는 K 를 고정하지만, Top-P는 분포 형태에 따라 고려하는 토큰 수를 동적으로 조절한다. 누적 확률이 P 가 될 때까지의 토큰만 고려한다.

```

1  class TopPSampler:
2      def __init__(self, p: float = 0.9, temperature: float = 1.0):
3          self.p = p
4          self.temperature = temperature
5
6      def __call__(self, logits):
7          scaled = logits / self.temperature
8          probs = F.softmax(scaled, dim=-1)
9          sorted_probs, sorted_indices = torch.sort(probs, descending=True, dim=-1)
10         cumulative = torch.cumsum(sorted_probs, dim=-1)
11         # P 넘는 위치부터 마스킹
12         sorted_mask = cumulative - sorted_probs > self.p
13         sorted_probs = sorted_probs.masked_fill(sorted_mask, 0.0)
14         sorted_probs = sorted_probs / sorted_probs.sum(dim=-1, keepdim=True).clamp(min=1e-10)
15         sampled = torch.multinomial(sorted_probs, num_samples=1).squeeze(-1)
16         return sorted_indices.gather(-1, sampled.unsqueeze(-1)).squeeze(-1)

```

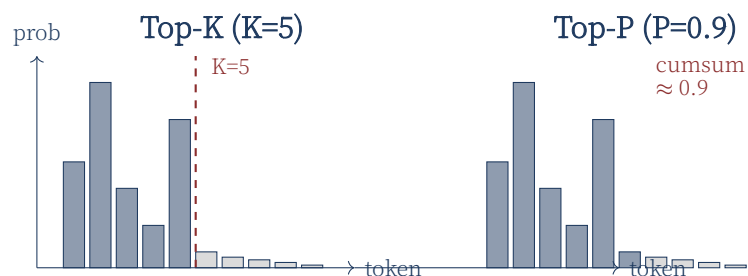


Figure 8.3: Top-K vs Top-P. Top-K는 고정 개수를, Top-P는 누적 확률이 P 가 될 때까지 선택.

8.5.1 Top-K vs Top-P

	Top-K	Top-P (Nucleus)
고려 토큰 수	고정 (K)	동적 (분포에 따라)
분포가 날카로울 때	너무 많은 토큰 고려	적은 토큰만 고려 (자동)
분포가 평평할 때	너무 적게 고려	많은 토큰 고려 (자동)
적응성	낮음	높음

Top-P가 보통 더 자연스러운 결과를 낸다. 모델이 확신할 때는 적은 토큰만, 불확실할 때는 많은 토큰을 고려하기 때문이다.

8.6 샘플링 전략 비교

전략	다양성	일관성	사례
Greedy	낮음	높음	코드 생성, 수학
Temperature ($T=0.7$)	중간	중간	일반적 용도
Top-K ($K=50$)	중간	높음	챗봇
Top-P ($P=0.9$)	중간	높음	챗봇, 창작
Top-P ($P=0.95$)	높음	중간	창의적 글쓰기

8.7 생성 함수 구현

모든 샘플러를 결합한 `generate` 함수를 만든다.

```

1 @torch.no_grad()
2 def generate(model, tokenizer, prompt, max_new_tokens=50,
3             sampler=None, device="cpu", stop_on_eos=True):
4     if sampler is None:
5         sampler = GreedySampler()
6     model.eval()
7     model.to(device)
8     ids = tokenizer.encode(prompt, add_bos=True, add_eos=False)
9     input_ids = torch.tensor([ids], dtype=torch.long, device=device)
10    eos_id = tokenizer.special.eos_id
11    context_length = model.config.context_length
12
13    for _ in range(max_new_tokens):
14        # 컨텍스트 길이 제한
15        x = input_ids[:, -context_length:] if input_ids.shape[1] > context_length else input_ids
16        logits = model(x)
17        next_logits = logits[:, -1, :] # 마지막 위치
18        next_id = sampler(next_logits)
19        input_ids = torch.cat([input_ids, next_id.unsqueeze(0)], dim=1)
20        if stop_on_eos and next_id.item() == eos_id:
21            break
22    return tokenizer.decode(input_ids[0].tolist())

```

8.8 컨텍스트 길이 제한

모델은 학습 시 정해진 `context_length`까지만 처리할 수 있다. 생성 도중 시퀀스가 이 길이를 초과하면 가장 오래된 토큰을 잘라낸다 (sliding window). 이는 근사치지만 작동한다. 2권에서는 이 한계를 극복하는 RoPE의 extrapolation 기법을 다룬다.

Sliding Window의 문제

컨텍스트를 잘라내면 모델이 “앞부분”을 잊어버린다. “Once upon a time, there was a princess named Aurora. She lived in a castle. One day, [긴 설명] ... Aurora decided to”에서 앞부분을 잘라내면 “Aurora”가 공주 이름이라는 맥락이 사라진다.

해결책: 더 긴 컨텍스트 모델 사용 (RoPE extrapolation), 또는 요약하여 압축. 2권에서 다룬다.

8.9 샘플러 테스트

```
1 def test_greedy_picks_max():
2     sampler = GreedySampler()
3     logits = torch.tensor([[1.0, 3.0, 2.0, 0.5]])
4     out = sampler(logits)
5     assert out.item() == 1 # 가장 큰 값
6
7 def test_top_k_limits_choices():
8     torch.manual_seed(0)
9     logits = torch.tensor([[1.0, 5.0, 4.0, 3.0, 0.1]])
10    sampler = TopKSampler(k=2, temperature=1.0)
11    for _ in range(20):
12        idx = sampler(logits).item()
13        assert idx in (1, 2) # 항상 top-2 중 하나
14
15 def test_top_p_limits_choices():
16     torch.manual_seed(0)
17     logits = torch.tensor([[0.0, 10.0, 9.0, 0.0, 0.0]])
18     sampler = TopPSampler(p=0.5, temperature=1.0)
19     for _ in range(20):
20         idx = sampler(logits).item()
21         assert idx == 1 # 가장 확률 높은 토큰만
```

8.10 실습: 다양한 샘플링 비교

`scripts/tiny_train.py` 데모를 실행하면 5가지 샘플링 전략의 출력을 비교할 수 있다.

```
cd make-llm-basic
python scripts/tiny_train.py --epochs 5 --vocab 500
```

출력 예:

```
1 프롬프트
2 : 'the'
3 [Greedy           ] the cat sat on the mat
4 [Temperature=0.5 ] the lazy dog sleeps
5 [Temperature=1.0  ] the brown fox jumps
```

6	[Top-K=10	] the quick fox runs
7	[Top-P=0.9	] the warm sun shines

8.11 고급 디코딩 기법 (소개)

이 책에서 구현하지는 않지만, 실제 시스템에서 쓰이는 고급 기법들을 소개한다:

8.11.1 Beam Search

각 스텝에서 K 개의 후보 시퀀스를 유지하며, 마지막에 가장 높은 점수의 시퀀스를 선택. 번역, 요약 등에 적합하지만, 챗봇에는 부적합 (다양성 부족).

8.11.2 Speculative Decoding

작은 “초안” 모델이 먼저 K 개 토큰을 생성하면, 큰 모델이 병렬로 검증. 맞으면 K 개를 한 번에 채택, 틀리면 처음부터. 처리량 2~3배 향상 가능.

8.11.3 Contrastive Search

반복을 피하면서도 일관성을 유지하는 최신 기법. 이전 토큰 표현과의 유사도로 패널티를 주어 다양한 토큰을 선택.

8.12 요약

- 자기회귀 생성은 한 번에 토큰 하나씩 생성하며 시퀀스를 확장.
- Greedy: 가장 확률 높은 토큰. 단조로움, 반복 루프 위험.
- Temperature: 분포의 날카로움을 조절. T 낮을수록 결정론적.
- Top-K: 상위 K 개만 고려. 간단하지만 비적응적.
- Top-P: 누적 확률 P 까지의 토큰만 고려. 분포에 적응적.
- 컨텍스트 길이를 초과하면 가장 오래된 토큰을 잘라냄.
- Speculative decoding, beam search 등 고급 기법도 존재.

8.13 연습 문제

1. Greedy 디코딩이 반복 루프에 빠지는 이유를 분석하라. 어떤 분포일 때 루프가 잘 생기는가?
2. Temperature가 0에 가까워지면 greedy와 어떻게 다른가? 0이면 어떤 일이 일어나는가?
3. Top-K와 Top-P를 동시에 적용하면 어떤 효과가 있는가? (즉, Top-K 후보 중에서 Top-P 필터링)
4. 챗봇에는 어떤 샘플링이 가장 적합한가? 이유를 설명하라.
5. Speculative decoding이 빠른 이유는 무엇인가?

이것으로 1권의 핵심 구현이 끝났다. 다음 장에서는 전체를 정리하고, 2권에서 다룰 고급 주제를 소개한다.

Chapter 9

정리와 다음 단계

축하한다. 여기까지 따라왔다면 독자는 이제 토큰나이저부터 텍스트 생성까지 LLM의 모든 구성 요소를 직접 구현해 본 것이다. 이 장에서는 1권을 정리하고, 2권 Make LLM-advanced에서 다음 고급 주제를 소개한다.

9.1 1권 요약

9.1.1 구현한 것

장	구현 내용
3장	BPE, Unigram 토큰나이저 (학습/인코딩/디코딩)
4장	Token/Positional Embedding, EmbeddingStack
5장	Scaled Dot-Product Attention, Causal Mask, Multi-Head Attention
6장	FeedForward (GELU), Transformer Block (Pre-LN)
7장	GPT 모델, LMDataset, Cross-Entropy, AdamW, Cosine LR, Trainer
8장	Greedy/Temperature/Top-K/Top-P 샘플러, generate 함수

9.1.2 테스트

총 48개의 단위 테스트가 모두 통과한다. 형상 테스트, 수치 테스트, 통합 테스트를 통해 모든 모듈이 올바르게 동작함을 검증했다.

```
$ pytest tests/ -v
===== 48 passed in 2.47s =====
```

9.1.3 독자가 가진 것

이제 독자는 다음을 할 수 있다:

- 자신만의 말뭉치로 토큰나이저 학습
- 임의의 크기 GPT 모델 생성 (config만 바꾸면 됨)
- CPU/GPU에서 학습 루프 실행
- 다양한 샘플링 전략으로 텍스트 생성
- HuggingFace Transformers의 코드를 읽을 때 내부 동작 이해

9.2 1권의 한계

솔직히 말하면, 1권에서 만든 모델은 ChatGPT와는 거리가 멀다. 한계는 크게 세 가지다.

크기: 1권 모델은 1천만~1억 파라미터 수준이다. GPT-3는 1,750억, GPT-4는 추정 1.7조 파라미터다. 이 크기 차이를 극복하려면 분산 학습이 필수다.

아키텍처: 1권은 2017년 원본 트랜스포머에 가깝다. 현대 LLM(LLaMA, Mistral, GPT-4)은 RoPE, GQA, SwiGLU, RMSNorm 등 더 효율적이고 안정적인 컴포넌트를 사용한다.

정렬: 1권 모델은 그냥 “다음 토큰 예측”만 학습했다. 사용자의 질문에 답하는 챗봇으로 만들려면 SFT(Supervised Fine-Tuning), RLHF(Reinforcement Learning from Human Feedback), DPO(Direct Preference Optimization) 같은 정렬 기법이 필요하다.

9.3 2권에서 다룰 것

2권 Make LLM-advanced는 위 세 가지 한계를 극복하는 법을 다룬다.

9.3.1 분산 학습 (2~4장)

- **DDP (Distributed Data Parallel):** 여러 GPU에서 같은 모델 복제 학습.
- **FSDP (Fully Sharded Data Parallel):** 모델을 샤드로 분할하여 메모리 절약.
- **ZeRO-1/2/3:** 옵티마이저/그래디언트/파라미터를 샤딩.
- **Pipeline Parallel:** 모델을 층별로 다른 GPU에 배치.
- **Tensor Parallel:** 단일 레이어를 여러 GPU로 분할 (Megatron-LM).

9.3.2 최신 아키텍처 (5~6장)

- **RoPE (Rotary Position Embedding):** 복소수 회전으로 위치 인코딩. extrapolation 우수.
- **GQA (Grouped Query Attention):** KV 헤드를 줄여 추론 메모리 절약.
- **SwiGLU:** GLU 변종으로 FFN 성능 향상. LLaMA가 사용.
- **RMSNorm:** LayerNorm의 경량화 버전.
- **MoE (Mixture-of-Experts):** 전문가 네트워크 풀에서 토큰별 상위 K개만 활성화.

9.3.3 양자화와 추론 최적화 (7장)

- **INT8/INT4 양자화:** 가중치를 저비트 정수로 변환하여 메모리 4~8배 절약.
- **AWQ:** 활성값 통계를 활용한 정확도 손실 최소화.
- **GPTQ:** 2차 정보를 사용한 양자화.
- **KV Cache:** 이전 토큰의 K, V를 재사용하여 추론 가속.
- **PagedAttention:** vLLM의 페이지 단위 KV 관리로 메모리 단편화 해결.

9.3.4 정렬과 파인튜닝 (8장)

- **SFT (Supervised Fine-Tuning):** 지도학습으로 챗봇 행동 학습.
- **RLHF:** 보상 모델 + PPO로 인간 선호도 최적화.

- **DPO**: RLHF의 복잡함 없이 직접 선호도 최적화.
- **LoRA**: 저랭크 어댑터로 효율적 파인튜닝.

9.3.5 데이터 파이프라인 (9장)

- **품질 필터**: 길이, 언어, 반복 비율로 필터링.
- **MinHash 중복 제거**: Jaccard 유사도 근사로 중복 문서 탐지.
- **합성 데이터**: 템플릿 기반 학습 데이터 생성.

9.4 추천 학습 경로

2권으로 넘어가기 전에 추천하는 활동:

1. **작은 프로젝트**: 1권 코드로 자신의 관심 분야 텍스트(예: 위키백과 한국어, GitHub 코드)를 학습해보라. 10만 토큰 정도면 1시간 안에 학습된다.
2. **논문 읽기**: “Attention Is All You Need”(Vaswani et al. 2017), “Language Models are Few-Shot Learners”(Brown et al. 2020)를 읽어보라. 1권을 읽었으므로 훨씬 이해가 잘 될 것이다.
3. **HuggingFace 코드 읽기**: `transformers/models/gpt2/modeling_gpt2.py`를 열어보라. 우리가 만든 코드와 얼마나 비슷한지, 어떤 차이가 있는지 비교하라.

9.5 마지막으로

LLM은 복잡해 보이지만, 결국 토큰라이저, 임베딩, 어텐션, FFN, 학습 루프의 조합이다. 이 기본기를 익히면 어떤 새로운 아키텍처, 어떤 새로운 최적화 기법이 나와도 이해할 수 있다. 2권에서는 이 기반 위에 ChatGPT 수준의 기술을 쌓아올릴 것이다.

코딩을 멈추지 마라. 실험을 멈추지 마라. 그것이 실력을 키우는 유일한 길이다.

1권 끝
dirmich

Appendix A

수학 기초 보충

이 부록에서는 본문에서 사용한 수학적 개념을 보충 설명한다. 미적분과 기초 선형대수를 알지만 행렬 미분 등에 익숙하지 않은 독자를 위한 참고 자료다.

A.1 벡터와 행렬

벡터는 숫자의 1차원 배열이다. $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ 로 표기. **행렬**은 2차원 배열. $A \in \mathbb{R}^{m \times n}$ 은 m 행 n 열.

- **내적(dot product)**: $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$. 스칼라 반환.
- **행렬곱**: $C = AB$ where $C_{ij} = \sum_k A_{ik} B_{kj}$.
- **전치(transpose)**: A^T 는 행과 열을 바꿈.
- **norm**: $\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$. 벡터의 길이.

A.2 Softmax

Softmax는 임의의 실수 벡터를 확률 분포로 변환하는 함수다.

Softmax

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

출력의 모든 원소는 0과 1 사이이며, 합은 1이다. 어텐션 가중치, 분류 모델의 출력 등에 사용.

A.3 Cross-Entropy

교차 엔트로피(cross-entropy)는 두 확률 분포 p, q 사이의 차이를 측정한다.

Cross-Entropy

$$H(p, q) = - \sum_i p_i \log q_i$$

분류 문제에서 p 는 정답 원-핫 벡터, q 는 모델 예측 softmax 출력. 단일 정답 y 에 대해 $H = -\log q_y$ 로 단순화.

A.4 연쇄 법칙과 Backpropagation

연쇄 법칙(chain rule)은 합성 함수의 미분법이다.

$$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$$

신경망은 합성 함수의 연속이므로, 연쇄 법칙을 통해 입력에서 출력까지의 미분을 계산할 수 있다. 이것이 **backpropagation**의 수학적 기반이다.

PyTorch는 이 과정을 자동화(autograd)하므로, 우리가 직접 미분 식을 유도할 필요는 없다. 하지만 “어떤 파라미터가 손실에 얼마나 영향을 미치는가”를 이해하려면 연쇄 법칙의 개념은 알아야 한다.

A.5 행렬 미분

손실 $\mathcal{L}$ 이 행렬 W 에 의존할 때, $\partial\mathcal{L}/\partial W$ 는 W 와 같은 shape의 행렬이며, 각 원소는 $\partial\mathcal{L}/\partial W_{ij}$ 다.

Jacobian은 벡터 함수의 미분을 나타내는 행렬이다. $\mathbf{y} = f(\mathbf{x})$ 일 때 $J_{ij} = \partial y_i / \partial x_j$.

자세한 내용은 “The Matrix Cookbook”(Petersen & Pedersen)을 참고하라.

A.6 정규 분포

정규 분포(normal distribution) $\mathcal{N}(\mu, \sigma^2)$ 의 확률 밀도 함수:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

가중치 초기화에 자주 사용된다. $\mu = 0, \sigma = 1/\sqrt{d}$ 를 쓰면 입력 차원이 커져도 출력 분산이 일정하게 유지된다 (Xavier 초기화의 기본 아이디어).

A.7 정보 이론 기초

엔트로피(entropy)는 확률 분포의 “무작위성”을 측정한다.

$$H(p) = -\sum_i p_i \log p_i$$

KL 발산(KL divergence)은 두 분포의 차이:

$$D_{KL}(p||q) = \sum_i p_i \log \frac{p_i}{q_i}$$

Cross-entropy는 $H(p) + D_{KL}(p||q)$ 로 분해된다. $H(p)$ 가 고정이므로 cross-entropy를 최소화하는 것은 D_{KL} 을 최소화하는 것과 같다.

A.8 최적화 기초

경사 하강법(gradient descent): 손실을 줄이는 방향으로 파라미터를 갱신.

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

η 는 학습률. **Adam**은 모멘텀과 적응적 학습률을 결합한 최적화 알고리즘이다. **AdamW**는 Adam에 weight decay를 결합한 버전으로 LLM 학습의 표준이다.

자세한 유도는 Goodfellow et al. “Deep Learning” 4~8장을 참고하라.

Appendix B

코드 전체 목록

이 부록은 1권에서 구현한 핵심 모듈의 전체 코드를 모은 것이다. 본문에서는 설명을 위해 부분적으로 발췌했지만, 여기서는 완전한 코드를 제공한다. 최신 버전은 GitHub 저장소에서 확인하라.

B.1 프로젝트 구조

```
make-llm-basic/  
  src/make-llm/  
    __init__.py  
  tokenizer/  
    __init__.py  
    base.py          # Tokenizer 추상 클래스  
    bpe.py           # BPE 토큰나이저  
    unigram.py       # Unigram 토큰나이저  
  model/  
    __init__.py  
    embedding.py     # TokenEmbedding, PositionalEmbedding  
    attention.py     # MultiHeadAttention, CausalMask  
    transformer_block.py  
    gpt.py           # GPT 모델  
  training/  
    __init__.py  
    dataset.py       # LMDataset  
    loss.py          # cross_entropy_loss  
    optimizers.py    # AdamW + cosine scheduler  
    trainer.py       # Trainer 클래스  
  inference/  
    __init__.py  
    sampler.py       # Greedy, Top-K, Top-P, Temperature  
    generate.py       # generate 함수  
  utils/  
    __init__.py  
    config.py        # ModelConfig, TrainerConfig  
    seed.py          # set_seed  
    logging.py       # Logger  
  tests/             # pytest 단위 테스트  
  book/              # 이 책의 LaTeX 소스
```

B.2 테스트 실행

```
# 패키지 설치
```

```

cd make-llm-basic
pip install -e .

# 전체 테스트 실행
pytest tests/ -v

# 특정 모듈만 테스트
pytest tests/test_tokenizer.py -v
pytest tests/test_model.py -v
pytest tests/test_training.py -v

```

B.3 빠른 시작 예제

```

1  """권의1 모든 모듈을 사용한 end-to-end 예제."""
2  from makellm.tokenizer import BPETokenizer
3  from makellm.model import GPT
4  from makellm.training import LMDataset, Trainer
5  from makellm.inference import generate, TopPSampler
6  from makellm.utils import ModelConfig, TrainerConfig, set_seed
7
8  set_seed(42)
9
10 # 1. 토큰나이저 학습
11 corpus = ["the quick brown fox jumps", "the lazy dog sleeps"] * 100
12 tokenizer = BPETokenizer()
13 tokenizer.train(corpus, vocab_size=500)
14
15 # 2. 모델 생성
16 model_config = ModelConfig(
17     vocab_size=tokenizer.vocab_size,
18     context_length=32, d_model=64, n_heads=4, n_layers=2, d_ff=128
19 )
20 model = GPT(model_config)
21 print(f"Model: {model}")
22 print(f"Parameters: {model.num_parameters():,}")
23
24 # 3. 데이터셋 준비
25 dataset = LMDataset(corpus, tokenizer, context_length=32)
26
27 # 4. 학습
28 trainer_config = TrainerConfig(
29     batch_size=4, learning_rate=3e-4, max_steps=100,
30     warmup_steps=10, device="cpu"
31 )
32 trainer = Trainer(model, dataset, trainer_config, model_config)
33 trainer.train()
34
35 # 5. 텍스트 생성
36 sampler = TopPSampler(p=0.9, temperature=0.8)
37 text = generate(model, tokenizer, prompt="the", max_new_tokens=20, sampler=sampler)
38 print(f"Generated: {text}")

```

B.4 모듈별 API 요약

모듈	주요 API
makellm.tokenizer	BPETokenizer, UnigramTokenizer
makellm.model	GPT, MultiHeadAttention, TransformerBlock
makellm.training	LMDataset, Trainer, cross_entropy_loss
makellm.inference	generate, GreedySampler, TopKSampler, TopPSampler
makellm.utils	ModelConfig, TrainerConfig, set_seed

B.5 설정 데이터클래스

```
1 @dataclass
2 class ModelConfig:
3     vocab_size: int = 1000
4     context_length: int = 128
5     d_model: int = 128
6     n_heads: int = 4
7     n_layers: int = 4
8     d_ff: int = 512
9     dropout: float = 0.1
10    use_bias: bool = True
11    layer_norm_eps: float = 1e-5
12    pad_token_id: int = 0
13
14 @dataclass
15 class TrainerConfig:
16     batch_size: int = 32
17     learning_rate: float = 3e-4
18     weight_decay: float = 0.1
19     max_epochs: int = 10
20     warmup_steps: int = 100
21     lr_scheduler: str = "cosine"
22     grad_clip: float = 1.0
23     log_every: int = 10
24     eval_every: int = 200
25     save_every: int = 1000
26     seed: int = 42
27     device: str = "cpu"
28     out_dir: str = "./runs"
```

이 설정들을 조정하여 다양한 크기의 모델을 실험할 수 있다. 예를 들어 GPT-2 small과 비슷한 설정은 `d_model=768`, `n_heads=12`, `n_layers=12`, `context_length=1024`다 (다만 실제 학습에는 상당한 컴퓨팅 자원이 필요하다).

B.6 라이선스

코드는 MIT 라이선스로 배포된다. 자유롭게 복사, 수정, 상업적 사용이 가능하다. 책 본문은 CC BY-NC-SA 4.0 (저자 확인 필요).

Appendix C

자주 묻는 질문과 함정

이 부록은 1권을 학습하며 독자가 자주 겪는 문제와 질문을 정리한 것이다. 각 항목은 실제 구현 중 발생할 수 있는 구체적 상황을 다룬다.

C.1 환경 관련

C.1.1 Q: PyTorch 설치 후 CUDA가 인식되지 않아요

A: 두 가지를 확인하라. 첫째, NVIDIA 드라이버가 설치되어 있는지 (`nvidia-smi` 실행). 둘째, PyTorch 버전과 CUDA 버전이 호환되는지. 예를 들어 CUDA 12.1용 PyTorch를 CUDA 11.8 시스템에 설치하면 인식 안 됨.

```
# CUDA 버전 확인
nvidia-smi | grep "CUDA Version"

# PyTorch 재설치 (CUDA 용12.1)
pip uninstall torch
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

C.1.2 Q: 학습이 너무 느려요

A: 다음을 점검하라:

1. GPU를 사용하고 있는지 (`device="cuda"` 설정).
2. 배치 크기가 너무 작지 않은지 (GPU 활용도 낮음).
3. 데이터 로딩이 병목인지 (`num_workers` 증가).
4. 혼합 정밀도(BF16)를 사용하고 있는지.

C.2 토큰나이저 관련

C.2.1 Q: BPE로 학습한 토큰나이저가 “안녕”을 이상하게 토큰화해요

A: 한국어가 코퍼스에 충분히 포함되지 않았기 때문. BPE는 빈도 기반이라 희귀 문자는 바이트로 분해된다. 해결책:

- 한국어 코퍼스 비율을 높일 것.
- 어휘 크기를 늘릴 것 (한국어 음절 약 2만 개 + 서브워드).
- SentencePiece 같은 전문 토큰나이저 사용.

C.2.2 Q: encode/decode 라운드트립이 실패해요

A: 공백 처리를 확인하라. BPE의 “_” 문자가 디코딩 시 공백으로 변환되지 않을 수 있다. 우리의 구현에서는 `_postprocess` 메서드가 이를 담당한다:

```
1 def _postprocess(self, text):
2     text = text.replace("_", " ")
3     return text.strip()
```

C.2.3 Q: 특수 토큰 ID가 바뀌면 어떻게 되나요

A: 치명적. 모델은 `<pad>=0`, `<bos>=1`, `<eos>=2`, `<unk>=3`을 가정하고 학습됐다. 토큰라이저를 다시 학습하면서 특수 토큰 ID가 바뀌면, 모델 가중치가 더 이상 유효하지 않다. 항상 특수 토큰 ID를 고정하라.

C.3 모델 관련

C.3.1 Q: 어텐션 출력이 NaN이 나와요

A: 세 가지 원인이 흔하다:

1. **소프트맥스 포화:** 로짓이 너무 커서 softmax가 0/1로 포화. 스케일링 ($\sqrt{d_k}$) 확인.
2. **학습률 너무 큼:** 그래디언트 폭발. 학습률을 1/10으로 줄여보라.
3. **FP16 오버플로우:** 활성값이 FP16 범위(± 65504)를 초과. BF16으로 전환.

```
1 # NaN 확인
2 if torch.isnan(loss):
3     print("NaN detected! Check learning rate and gradients.")
4     print(f"Max grad: {max(p.grad.abs().max() for p in model.parameters() if p.grad is not None)}")
```

C.3.2 Q: 모델이 같은 단어를 반복해요

A: 샘플링 전략을 점검하라. Greedy는 반복 루프에 빠지기 쉽다. Top-P ($P=0.9$)로 전환하거나 Temperature를 0.7~0.9로 설정. 반복 패널티 (repetition penalty) 추가도 효과적.

```
1 # 반복 패널티 단순화()
2 def apply_repetition_penalty(logits, generated_ids, penalty=1.2):
3     for token_id in set(generated_ids.tolist()):
4         logits[token_id] /= penalty
5     return logits
```

C.3.3 Q: 컨텍스트 길이를 초과하면 에러가 나요

A: 모델의 `context_length`를 초과하는 입력을 넣으면 임베딩 단계에서 에러. `generate` 함수에서 슬라이딩 윈도우로 잘라내는 것을 확인하라:

```
1 if input_ids.shape[1] > context_length:
2     x = input_ids[:, -context_length:] # 가장 최근 것만 유지
```

C.4 학습 관련

C.4.1 Q: 손실이 줄어들이지 않아요

A: 다음을 점검하라:

1. 학습률이 너무 작거나 ($1e-6$ 이하) 너무 큼 ($1e-2$ 이상).
2. 데이터가 너무 적음 (최소 수천 토큰 필요).
3. 옵티마이저가 잘못됨 (AdamW의 beta, eps 확인).
4. 그래디언트가 흐르지 않음 (requires_grad 확인).

```
1 # 그래디언트 흐름 확인
2 for name, p in model.named_parameters():
3     if p.requires_grad and p.grad is None:
4         print(f"No gradient for {name}")
5     elif p.grad is not None and p.grad.abs().max() < 1e-10:
6         print(f"Vanishing gradient for {name}")
```

C.4.2 Q: 학습 초기에 손실이 발산해요

A: Warmup이 부족하거나 초기화가 잘못됐을 가능성. 다음을 시도:

- Warmup 스텝 늘리기 (100~1000).
- 그래디언트 클리핑 확인 (grad_clip=1.0).
- 초기화 표준편차 줄이기 ($0.02 \rightarrow 0.01$).

C.4.3 Q: 체크포인트를 로드하면 다른 결과가 나와요

A: 두 가지 원인:

1. **시드 미설정**: set_seed(42)를 호출했는지 확인.
2. **드롭아웃**: 모델을 eval() 모드로 전환했는지 확인.
3. **CuDNN 비결정론**: torch.backends.cudnn.deterministic = True.

C.5 분산 학습 관련 (2권 미리보기)

C.5.1 Q: DDP로 학습하면 속도가 오히려 느려요

A: 통신 병목일 가능성. 다음을 점검:

- 배치 크기가 너무 작아 연산/통신 비율이 낮음.
- NVLink가 아닌 느린 통신 (PCIe 등) 사용.
- 그래디언트 동기화가 매 스텝 일어남 (gradient accumulation으로 줄이기).

C.5.2 Q: FSDP에서 OOM이 나요

A: 활성화 메모리가 부족. 활성화 체크포인트링을 켜거나 배치 크기를 줄이기. CPU offload도 고려.

C.6 디버깅 팁

C.6.1 단계별로 검증하라

복잡한 버그는 작은 단위로 쪼개서 검증. 토크나이저만 단독 테스트, 어텐션만 단독 테스트 등.

```
1 # 토크나이저 단독 테스트
2 tok = BPETokenizer()
3 tok.train(["hello world"], vocab_size=50)
4 print(tok.encode("hello"))
5 print(tok.decode(tok.encode("hello")))
6
7 # 어텐션 단독 테스트
8 attn = MultiHeadAttention(config)
9 x = torch.randn(1, 4, 32)
10 out = attn(x)
11 print(out.shape, out.dtype, torch.isnan(out).any())
```

C.6.2 shape를 항상 출력하라

버그의 80%는 shape 불일치. 모든 단계에서 shape를 출력하는 습관.

```
1 print(f"input shape: {x.shape}")
2 print(f"after embedding: {emb.shape}")
3 print(f"after attention: {attn_out.shape}")
```

C.6.3 작은 입력으로 시작하라

배치 1, 시퀀스 4로 시작해서 동작을 확인한 후 크기를 키울 것. 큰 입력에서 디버깅하면 출력이 너무 많아 혼란.

C.7 성능 최적화 팁

C.7.1 혼합 정밀도 사용

```
1 from torch.cuda.amp import autocast, GradScaler
2
3 scaler = GradScaler()
4 with autocast(dtype=torch.bfloat16):
5     logits = model(inputs)
6     loss = criterion(logits, targets)
7 scaler.scale(loss).backward()
8 scaler.step(optimizer)
9 scaler.update()
```

BF16은 FP16과 달리 scaler가 필요 없다. A100 이상에서 권장.

C.7.2 torch.compile

PyTorch 2.0의 torch.compile로 모델을 컴파일하면 1.5~2배 속도 향상.

```
1 model = GPT(config)
2 model = torch.compile(model) # 컴파일
```

C.7.3 프로파일링

```
1 with torch.profiler.profile(activities=[torch.profiler.ProfilerActivity.CPU,  
2                               torch.profiler.ProfilerActivity.CUDA]) as prof:  
3     # 학습 코드  
4 prof.export_chrome_trace("trace.json")
```

Chrome 브라우저에서 `chrome://tracing`으로 열어 병목 분석.

Appendix D

연습 문제 해답

이 부록은 본문 연습 문제의 해답을 제공한다. 스스로 풀어본 후 확인하기 바란다.

D.1 1장 해답

1-1. ChatGPT의 답변을 평가해보라.

모범 답변: “언어 모델은 다음 토큰 예측으로 학습되지만, 충분히 큰 모델과 다양한 데이터를 사용하면 질문-답변 패턴을 암묵적으로 학습한다. 사용자가 질문 형태로 입력하면, 모델은 학습 데이터에서 그 질문 다음에 오던 답변 패턴을 생성한다.”

1-2. “태양은 동쪽에서 ...” 다음 단어 확률.

사람이라면 “쬡는”을 99% 이상으로 매긴다. 이유: (1) 일상적 관찰, (2) 언어적 콜로케이션 (“동쪽에서 쬡는”은 고정 표현), (3) 문맥의 강한 제약.

1-3. N-gram의 희소성 문제.

“북극곰이 춤을” 다음 단어를 N-gram이 예측하려면 학습 데이터에 “북극곰이 춤을” 다음 단어가 등장했어야 한다. 이 3-gram이 학습되지 않았으면 확률이 0이 되어, smoothing으로 “더” 같은 일반적 단어를 추천하게 된다.

1-4. 트랜스포머가 RNN보다 병렬 처리에 유리한 이유.

RNN은 시점 t 의 계산이 시점 $t - 1$ 의 은닉 상태에 의존하므로 순차 처리 필수. 트랜스포머는 어텐션으로 모든 위치를 동시에 처리하므로 GPU 병렬화에 최적.

1-5. Chinchilla 기준 데이터 스케일.

파라미터를 10배 키우면 데이터도 약 10배 키워야 (20 토큰/파라미터 비율 유지). 즉 10배 모델은 200배 데이터 필요.

D.2 3장 해답

3-1. 빈도가 낮은 쌍을 병합하면?

자주 등장하지 않는 패턴이 토큰이 되어 어휘가 비효율적. 희귀한 “xz” 같은 쌍이 병합되면 실제 텍스트에서 거의 사용되지 않는 토큰이 됨.

3-2. Unigram 패널티 -5.0으로 변경.

어휘에 없는 부분문자열의 패널티가 작아지므로, Viterbi가 더 적극적으로 분할을 시도. 결과적으로 더 많은 작은 토큰으로 분해되는 경향.

3-3. 한국어 BPE, 어휘 100 vs 1000.

어휘 100: “안녕하세요”가 5개 이상 바이트 토큰으로 분해. 매우 비효율적. 어휘 1000: “안녕”, “하세요” 같은 서브워드 학습 가능. 시퀀스 길이 대폭 단축.

3-4. Viterbi 길이 상한 4로 축소.

5자 이상의 토큰이 사라짐. “hello” 같은 단어가 “he”, “ll”, “o”로 분해되어 시퀀스 길이 증가.

3-5. <bos>의 필요성.

없어도 모델은 동작하지만, 시퀀스 시작을 명시적으로 표시하면 모델이 “여기서부터 새 시퀀스”임을 인식. 배치 처리 시 여러 시퀀스를 구분하는 데도 유용.

D.3 4장 해답

4-1. 임베딩 차원이 너무 작거나 큰 경우.

작으면 (8): 단어 간 구분이 어려워 의미적 유사성 학습 불가. 크면 (4096): 파라미터 폭발, 메모리 부족, 과적합 위험. 보통 256~4096 사용.

4-2. Sinusoidal 상수 100 → 100.

주기가 짧아져 가까운 위치도 구분하기 어려워짐. 위치 인코딩의 “해상도”가 낮아짐.

4-3. 스케일링 생략.

위치 임베딩이 토큰 임베딩을 압도하여 모델이 위치에만 의존. 의미 정보 손실.

4-4. 학습 가능한 위치 임베딩, max_len 초과.

인덱스 범위를 벗어나 에러 발생. RoPE 같은 extrapolation이 가능한 방식이 필요.

4-5. LayerNorm ϵ 값 변경.

ϵ 이 크면 정규화가 약해져 출력 분산이 커짐. 너무 작으면 분모가 0에 가까워 수치 불안정.

D.4 5장 해답

5-1. 스케일링 생략 시 softmax.

$d_k = 64$ 일 때 내적 값이 ± 64 까지 갈 수 있어 softmax가 원-핫에 포화. 그래디언트 0, 학습 멈춤.

5-2. 인과 마스크 미사용.

학습 시: 모델이 미래 토큰을 “보고” 정답을 맞추므로 학습 손실은 낮아지지만 일반화 안 됨. 추론 시: 미래 토큰이 없으므로 학습-추론 갭 발생. 성능 급락.

5-3. 헤드 수 1 (단일 헤드).

하나의 어텐션 패턴만 학습. 문법, 의미, 담화 관계를 한 헤드가 모두 처리해야 하므로 성능 저하.

5-4. 헤드 수 = d_{model} ($d_{head} = 1$).

각 헤드가 1차원만 처리. 어텐션 의미가 사라지고 단순 가중합이 됨. 사실상 선형 변환과 동일.

5-5. 어텐션 행렬 메모리, $L = 100,000$.

$100000^2 \times 2 \text{ bytes (FP16)} = 20 \text{ GB}$. 단일 헤드 기준. 헤드가 32개면 640 GB. 사실상 불가능.

D.5 6장 해답

6-1. $d_{ff} = d_{model}$.

FFN의 확장-축소 패턴이 사라져 비선형 표현력 감소. 더 깊은 선형 변환과 유사.

6-2. 잔차 없이 12층.

backward 시 그래디언트가 앞쪽 층에 도달하지 못해 학습 불가. 손실이 줄어들지 않음.

6-3. Pre-LN에서 $\ln_f$ 필요성.

Pre-LN은 각 블록 입력을 정규화하지만 마지막 블록 출력은 정규화되지 않은 상태. LM Head에 넣기 전 정규화 필요.

6-4. GELU 대신 SiLU.

성능 차이 미미. SiLU ($x \cdot \sigma(x)$)도 부드러운 전환. 일부 모델(LLaMA의 FFN은 SwiGLU이지만 활성화로 SiLU 사용)이 채택.

6-5. 잔차 스케일링 없이 100층.

초기 활성화값 분산이 층마다 누적되어 폭발. 학습 초기에 NaN 발생 가능성 높음.

D.6 7장 해답

7-1. 가중치 타이 미사용, GPT-2 small.

$V \times d = 50257 \times 768 \approx 38.6M$ 파라미터 추가. 총 $124M \rightarrow 163M$.

7-2. Warmup 0.

학습 초기 그래디언트가 불안정하여 발산 가능성. 특히 Post-LN에서 심각. Pre-LN은 덜 민감하지만 여전히 권장.

7-3. Label smoothing 0.3.

강한 smoothing으로 모델이 정답에 대한 확신을 덜 가짐. 일반화는 향상되지만 학습 손실이 높게 유지. 보통 0.1이 적정.

7-4. 그래디언트 클리핑 없음.

그래디언트 폭발 시 가중치가 크게 변하여 모델이 붕괴. 특히 큰 학습률에서 위험.

7-5. PPL 1.0 가능?

이론적으로 불가능. 자연어는 본질적 불확실성이 있어 다음 토큰이 100% 예측 불가. 학습 데이터에 완전히 외운 경우에만 특정 문장에서 $PPL \approx 1$ 가능하지만 일반적이지 않음.

D.7 8장 해답

8-1. Greedy 반복 루프 원인.

분포가 “store” 다음 “to buy”를 높게, “to buy” 다음 “some store”를 높게 예측하는 루프가 형성되면 greedy는 거기서 빠져나오지 못함. 확률이 높은 경로만 따라가기 때문.

8-2. Temperature $\rightarrow 0$.

$T = 0$ 이면 softmax가 argmax가 되어 greedy와 동일. $T = 0.01$ 이면 거의 greedy. 분포가 너무

날카로워져 다양성 0.

8-3. Top-K 후 Top-P.

Top-K로 후보를 K개로 줄인 후, 그 중에서 누적 확률 P까지 선택. 더 강력한 필터링. 품질은 향상되지만 다양성 감소.

8-4. 챗봇에 적합한 샘플링.

Top-P ($P=0.9$, $T=0.7\sim0.9$). 다양성과 일관성의 균형. Greedy는 단조로움, 순수 샘플링은 일관성 부족.

8-5. Speculative decoding이 빠른 이유.

작은 모델이 K개 토큰을 빠르게 생성하면, 큰 모델이 그 K개를 한 번에 검증 (병렬 처리). 맞으면 K개를 한 스텝에 채택하므로 큰 모델의 스텝 수가 $1/K$ 로 감소.

Appendix E

고급 주제 미리보기

이 부록은 2권 Make LLM-advanced에서 다룰 고급 주제를 간략히 소개한다. 1권을 마친 독자가 다음 학습 방향을 잡는 데 도움이 된다.

E.1 분산 학습

1권 모델은 단일 GPU/CPU에서 학습했다. 하지만 70B 이상 모델은 단일 GPU 메모리에 들어가지 않는다.

DDP (Distributed Data Parallel): 모델 복제본을 각 GPU에 두고 데이터만 분할. 가장 단순.

FSDP (Fully Sharded Data Parallel): 파라미터, 그래디언트, 옵티마이저 상태를 모두 샤딩. 70B+ 모델 학습 가능.

ZeRO: DeepSpeed의 단계적 샤딩. ZeRO-1/2/3로 점진적 메모리 절약.

파이프라인 병렬: 모델을 층별로 GPU에 분할. 노드 간 통신에 적합.

텐서 병렬: 단일 레이어의 가중치를 분할. Megatron-LM 방식.

E.2 최신 아키텍처

1권은 2017년 원본 트랜스포머에 가깝다. 최신 LLM은 더 효율적인 컴포넌트를 사용한다.

RoPE (Rotary Position Embedding): 복소수 회전으로 위치 인코딩. extrapolation이 우수하여 긴 문맥 처리에 유리. LLaMA, Mistral 사용.

GQA (Grouped Query Attention): KV 헤드를 줄여 추론 메모리 절약. LLaMA-2 70B가 GQA-8 사용.

SwiGLU: GLU 변종 활성화로 FFN 성능 향상.

RMSNorm: LayerNorm의 경량 버전. 계산량 10~20% 감소.

MoE (Mixture-of-Experts): 여러 전문가 중 토큰별로 상위 K개만 활성화. Mixtral 8x7B가 47B 총 파라미터에 13B 활성화로 운영.

E.3 양자화와 추론 최적화

INT8/INT4 양자화: 가중치를 저비트 정수로 변환하여 메모리 4~8배 절약. 추론 비용 결정적 감소.

AWQ: 활성값 통계로 중요 채널 보존. INT4에서도 높은 정확도.

KV Cache: 자기회귀 생성에서 이전 K, V를 재사용. 매 스텝 복잡도 $O(L^2) \rightarrow O(L)$.

PagedAttention: vLLM의 페이지 단위 KV 관리. 다중 시퀀스 처리량 2~4배 향상.

E.4 정렬과 파인튜닝

1권 모델은 “다음 토큰 예측”만 한다. 챗봇으로 만들려면 정렬이 필요.

SFT (Supervised Fine-Tuning): 명령-응답 쌍으로 지도학습. 기본 챗봇 행동 학습.

RLHF: 보상 모델 + PPO로 선호도 최적화. ChatGPT가 사용.

DPO (Direct Preference Optimization): 보상 모델 없이 직접 선호도 최적화. 단순하고 효과적. Zephyr, Llama-2-Chat 사용.

LoRA: 저랭크 어댑터로 파인튜닝 비용 99% 절약. 7B 모델을 200MB 어댑터로 파인튜닝.

E.5 데이터 파이프라인

품질 필터: 길이, 언어, 반복 비율로 저품질 텍스트 제거.

MinHash: Jaccard 유사도 근사로 중복 문서 탐지. 웹 데이터의 30~50%가 중복.

합성 데이터: 강력한 모델로 특정 도메인 데이터 생성. Evol-Instruct, Self-Instruct 등.

E.6 2권 학습 준비

2권은 1권의 코드를 기반으로 확장한다. 1권의 GPT 모델을 가져와:

- 분산 학습으로 감싸고 (2~4장)
- 어텐션을 GQA로 교체하고 (5장)
- FFN을 SwiGLU로 바꾸고 (5장)
- LoRA 어댑터를 추가한다 (8장).

1권을 익숙하게 다루면 2권도 자연스럽게 따라갈 수 있다. 1권 코드를 직접 수정해보며 실험하는 것을 권장한다.

E.7 추천 학습 경로

1. 1권 코드로 작은 모델 학습 (scripts/tiny_train.py).
2. 자신의 말뭉치로 실험 (위키백과 한국어, GitHub 코드 등).
3. 2권으로 넘어가 분산 학습 환경 구축.
4. LoRA로 파인튜닝 실험 (scripts/lora_finetune.py).
5. 양자화로 추론 최적화 (scripts/quantize_demo.py).
6. 실제 HuggingFace 모델로 서빙 연습 (vLLM, TGI).

이 경로를 따라가면 ChatGPT 수준의 LLM을 이해하고 운영할 수 있는 능력을 갖추게 될 것이다. 코딩을 멈추지 마라. 실험을 멈추지 마라. 그것이 실력을 키우는 유일한 길이다.